

The Evolution of Distributed LLM Training Systems

Junan Chen
225040494

School of Data Science

The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

Recent years have witnessed the rapid development of large language models (LLMs), which have achieved remarkable performance across a wide range of natural language processing tasks. However, training these large models poses significant challenges due to their enormous memory and computational requirements, which far exceed the capacity of a single hardware accelerator. To address these challenges, a series of distributed training systems have been proposed to scale LLM training to thousands or even tens of thousands of GPUs. In this paper, we present a comprehensive survey and integration of three representative distributed LLM training systems, namely Megatron-LM, Alpa, and MegaScale, which represent the key milestones in the evolution of this field. We first introduce the fundamental background of distributed parallelism techniques, and then delve into the design, implementation, and evaluation of each system in detail. Megatron-LM pioneered the manual tensor parallelism approach, enabling the training of billion-parameter models with high scaling efficiency. Alpa further advanced the field by proposing an automated hierarchical parallelism framework, which unifies inter- and intra-operator parallelism and enables automatic parallelization for general models without manual tuning. MegaScale, the latest system in this evolution, achieves full-stack optimization for production-grade training, scaling to more than 10,000 GPUs with high efficiency and robust fault tolerance. Through this integration, we provide a holistic view of the evolution of distributed LLM training systems, from manual parallelism design to automated optimization, and from small-scale experiments to large-scale production deployment. We also discuss the future directions in this rapidly evolving field, aiming to inspire further research in scaling LLM training to even larger scales.

Contents

1	Introduction	6
1.1	Challenges	6
1.2	Overview of the Evolution	7
1.3	Research Contributions	8
2	Background and Related Work	10
2.1	Fundamental Parallelism Techniques	10
2.2	Related Work	11
3	Megatron-LM: Manual Tensor Parallelism for Large Model Training	13
3.1	Motivation	13
3.2	Design of Tensor Parallelism for Transformers	14
3.3	Architectural Adjustments for BERT Models	15
3.4	Experimental Results	17
3.4.1	Scalability Analysis	17
3.4.2	Language Modeling Results with GPT-2	19
3.4.3	BERT Results	19
3.5	Summary	20
4	Alpa: Automated Hierarchical Parallelism for Generalizable Training	22
4.1	Motivation	22
4.2	Hierarchical View of Parallelism	23
4.3	Compiler Passes	25
4.3.1	Intra-Operator Parallelism Pass	25
4.3.2	Inter-Operator Parallelism Pass	25
4.4	Runtime Orchestration	25
4.5	Experimental Results	27
4.5.1	End-to-End Performance	27
4.5.2	Ablation Studies	29
4.6	Summary	30
5	MegaScale: Full-Stack Optimization for 10K-GPU Production Training	31
5.1	Motivation	31
5.2	3D Parallelism Foundation	32
5.3	Full-Stack Efficiency Optimization	33

5.3.1	Algorithmic Optimizations	33
5.3.2	Communication Overlapping	34
5.3.3	Other System Optimizations	34
5.4	Fault Tolerance and Observability	35
5.4.1	Robust Training Workflow	35
5.4.2	In-Depth Observability and Diagnosis Tools	35
5.5	Production Experience and Results	36
5.6	Summary	39
6	Conclusion and Future Work	41
6.1	Summary of Contributions	41
6.2	Conclusion	42
6.3	Future Work	42

List of Figures

1.1	Trend of LLM parameter sizes.	7
3.1	Scaling of language model training. The figure shows that Megatron-LM enables training of much larger models with significantly reduced training time compared to previous systems.	15
3.2	Parallel blocks of Transformer with Model Parallelism. The figure shows how the MLP and self-attention blocks are parallelized across GPUs, with the f and g operators handling the all-reduce communication.	16
3.3	Communication operations in a transformer layer. The figure illustrates that there are only 4 total communication operations in the forward and backward pass of a single model parallel transformer layer, which minimizes the communication overhead.	16
3.4	Grouping of GPUs for hybrid model and data parallelism. The figure shows the logical grouping of GPUs, where model parallel groups are within nodes, and data parallel groups span across nodes.	17
3.5	Training loss for BERT model using the original architecture and the rearranged architecture. The left part shows that the original architecture fails to scale, while the rearranged architecture enables stable training with lower loss.	18
3.6	Model and model + data parallel weak scaling efficiency as a function of the number of GPUs. The system maintains high scaling efficiency even up to 512 GPUs.	18
3.7	Validation set perplexity. Larger language models converge noticeably faster and converge to lower validation perplexities than their smaller counterparts.	19
4.1	Parallelism techniques and their properties. The figure categorizes the existing parallelism techniques into intra-operator and inter-operator parallelism, showing their different communication and utilization properties.	24
4.2	Generation of parallelization plans for a computational graph. The figure shows how Alpa’s hierarchical approach combines inter- and intra-operator parallelism, enabling a much larger plan space than previous approaches.	24
4.3	Compiler passes and runtime architecture. The figure shows the workflow of the compiler, which first runs the inter-op pass to split the model into stages, then runs the intra-op pass to optimize each stage, and then the runtime orchestrates the execution.	26

4.4	Cross-mesh resharding optimization. The figure shows how Alpa optimizes the cross-mesh communication by using local all-gather to reduce the inter-node communication overhead.	26
4.5	Pipeline execution latency.	27
4.6	End-to-end evaluation results. The figure shows that Alpa outperforms the baselines on all models, especially on MoE and Wide-ResNet.	27
4.7	Cross-mesh resharding performance. The figure shows that the optimized resharding approach significantly outperforms the naive approach, reducing the communication time.	28
4.8	Parallelization plan for Wide-ResNet. The figure shows the automatically generated plan, which partitions the model into stages and assigns different mesh sizes to each stage.	28
4.9	Intra-operator parallelism ablation, showing that the ILP-based approach outperforms other baselines.	29
4.10	Inter-operator parallelism ablation, showing that the DP-based approach outperforms rule-based partitioning.	29
4.11	Compilation time of Alpa’s compiler. The compilation time scales linearly with the number of operators, enabling efficient handling of large models. . .	30
5.1	ZeRO data parallelism. The figure shows how the parameters are sharded across the data parallel dimension, reducing the memory requirement per worker.	33
5.2	Pipeline parallelism. The figure shows how the model layers are partitioned across the pipeline parallel dimension, enabling pipelined execution of micro-batches.	33
5.3	Overlapping communication in pipeline parallelism. The figure shows how the send and receive operations are overlapped with the forward and backward computation, hiding the communication overhead.	34
5.4	Overlapping communication in tensor parallelism. The figure shows how the communication is overlapped with the GEMM computation, hiding the communication overhead.	35
5.5	Robust training workflow. The figure shows how the system detects failures, runs diagnostics, evicts faulty nodes, and recovers the training from checkpoint.	36
5.6	CUDA event heat map. The heat map visualizes the execution time of each node, enabling the identification of straggler nodes.	37
5.7	Distributed timeline trace. The trace visualizes the execution of the pipeline across nodes, helping to identify pipeline bubbles and synchronization issues.	37
5.8	Inconsistent MFU across nodes. The figure shows that some nodes have lower MFU than others, indicating straggler issues.	37
5.9	Weak scaling results. The figure shows that the system maintains high MFU as the number of GPUs increases, up to 12,288 GPUs.	38
5.10	Micro-batch training loss. The loss converges smoothly, showing that the training is stable with the large batch size.	38
5.11	Stable MFU during production training. The MFU remains stable throughout the entire training run, showing the stability of the system.	39

List of Tables

3.1	Parameters used for scaling studies. Hidden size per attention head is kept constant at 96.	18
3.2	Model configurations used for GPT-2.	19
3.3	Zero-shot results on WikiText103 and LAMBADA.	20
3.4	Downstream task results for BERT models.	20
5.1	Strong-scaling training performance for the 175B model.	38

Chapter 1

Introduction

1.1 Challenges

The past decade has witnessed the explosive growth of large language models (LLMs), which have become the foundation of modern natural language processing (NLP) research and applications. Starting from small transformer models with hundreds of millions of parameters [30], the field has rapidly evolved to models with hundreds of billions or even trillions of parameters, such as GPT-3 [2], PaLM [4], and GPT-4 [21]. These large models have demonstrated unprecedented capabilities, including few-shot learning [2], complex reasoning [32], and cross-domain generalization, enabling a wide range of applications from chatbots to code generation and scientific research.

The scaling of model size has followed predictable patterns discovered through empirical studies. Research on scaling laws [14, 10] has established that model performance improves as a power law with respect to model size, dataset size, and computational budget. This insight has driven the development of increasingly large models, with recent foundation models like LLaMA [29] containing tens of billions of parameters and achieving remarkable zero-shot performance across diverse tasks. The Chinchilla optimal scaling study [10] further refined these findings, suggesting that model size and training tokens should scale proportionally for compute-optimal training.

However, the increasing size of these models also brings unprecedented challenges to the training infrastructure. As shown in Fig. 1.1, the parameter counts of Large Language Models have exploded from millions to hundreds of billions. A state-of-the-art 175 billion parameter model like GPT-3 requires approximately 350GB of memory just to store the parameters in fp16 precision, far exceeding the 32-80GB capacity of modern GPUs like the NVIDIA V100 or A100. The training process compounds this memory requirement by requiring additional memory for optimizer states, gradients, and activations, bringing the total memory demand to multiple terabytes per model replica.

The computational requirements are equally staggering. Training a 175 billion parameter model on 1 trillion tokens requires approximately 3.69×10^{23} FLOPs [14]. On a single NVIDIA V100 GPU with 125 TeraFLOPs of fp16 performance, this would take over 35,000 years. Even with 1,000 GPUs, the training time would exceed 35 years. These numbers underscore why distributed training across thousands of GPUs is not merely advantageous

but absolutely essential for training frontier models within practical timeframes.

The challenges extend beyond raw memory and compute requirements. At scale, communication bandwidth becomes a critical bottleneck, as gradient synchronization across thousands of GPUs must complete within each iteration. Network topology, congestion control, and collective communication algorithms all significantly impact training efficiency. Additionally, the mean time between failures (MTBF) for large GPU clusters means that training runs spanning weeks or months will inevitably experience hardware failures, network outages, and software bugs, requiring sophisticated fault tolerance mechanisms.

To address these challenges, the research community has developed a series of distributed training systems, which partition the model and computation across multiple devices, enabling the training of large LLMs on clusters of GPUs. These systems have evolved through distinct phases, each addressing the limitations of previous approaches and introducing new innovations that push the boundaries of scale and efficiency.

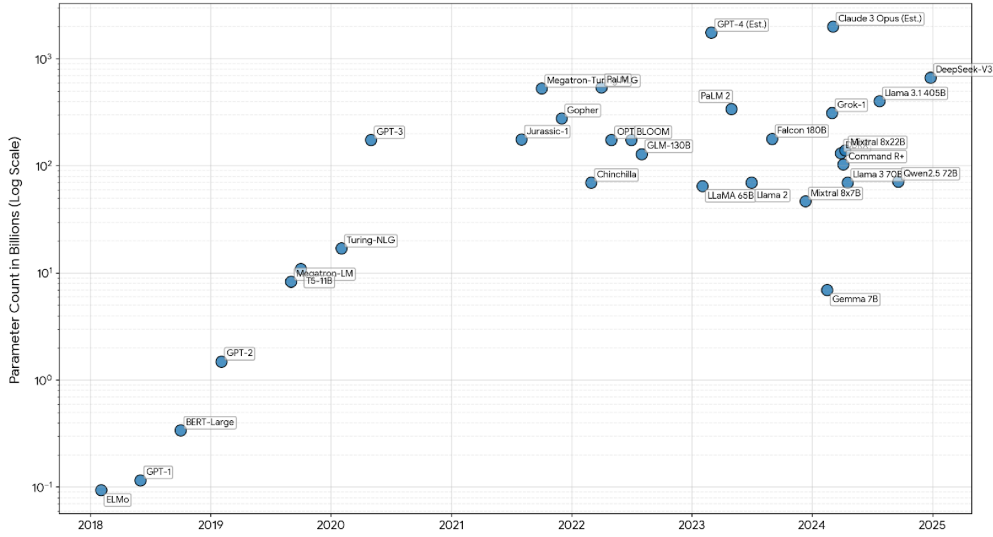


Figure 1.1: Trend of LLM parameter sizes.

1.2 Overview of the Evolution

The evolution of distributed LLM training systems has gone through several key stages, each addressing the limitations of the previous generation. In the early stage, researchers focused on developing manual parallelism techniques tailored specifically for transformer models, as the homogeneous architecture of transformers allowed for hand-designed parallelization strategies that could achieve high efficiency. Megatron-LM [26], one of the pioneering systems in this stage, introduced the intra-layer tensor parallelism approach, which partitions the computation of each transformer layer across multiple GPUs, enabling the training of models with up to 8.3 billion parameters on 512 GPUs with 76

As the model size continued to grow, and researchers started to train more heterogeneous models beyond the standard transformer architecture, the manual parallelism approach became increasingly burdensome. Designing a parallelization plan for a new model required

significant engineering effort and system expertise, and the hand-tuned parameters could not generalize to different cluster environments or model architectures. To address this limitation, researchers started to explore automated parallelization systems, which could automatically generate the optimal parallelization plan for a given model and cluster. Alpa [34], a representative system in this stage, proposed a hierarchical parallelism framework that unifies inter-operator and intra-operator parallelism. By decomposing the parallelization problem into two hierarchical levels, Alpa can automatically generate efficient parallel execution plans for general models, without any manual tuning. It achieves performance comparable to or even better than hand-tuned systems, while generalizing to heterogeneous models and cluster environments.

Most recently, as LLMs started to be deployed in production environments, the focus shifted to scaling the training to tens of thousands of GPUs, and addressing the practical challenges of large-scale production training, such as fault tolerance, stragglers, and network stability. MegaScale [13], a production-grade distributed training system, addresses these challenges through a full-stack algorithm-system co-design approach. It optimizes the training efficiency through communication overlapping, operator optimization, and network tuning, and implements a robust fault tolerance framework with in-depth observability tools, enabling stable training of LLMs on more than 10,000 GPUs. This system represents the latest milestone in the evolution of distributed LLM training, bringing the technology from research experiments to real-world production deployment.

1.3 Research Contributions

In this paper, we integrate these three representative systems into a comprehensive thesis, providing a holistic view of the evolution of distributed LLM training systems. We organize the thesis into six chapters. Following this introduction, Chapter 2 presents the background and related work, introducing the fundamental parallelism techniques and existing research in this field. Chapter 3 presents Megatron-LM, the pioneering manual tensor parallelism system, detailing its design, implementation, and experimental results. This chapter highlights Megatron-LM’s key contributions: the communication-efficient tensor parallelism design that achieves only four all-reduce operations per transformer layer, the pre-LN architecture that stabilizes large model training, and the demonstration of scaling laws that established larger models consistently achieve better performance. Chapter 4 introduces Alpa, the automated hierarchical parallelism system, discussing its compiler passes, runtime architecture, and evaluation. This chapter emphasizes Alpa’s primary contribution: the novel hierarchical view of parallelism that unifies inter-operator and intra-operator parallelism through a two-level compilation approach, enabling automatic parallelization for general models without manual tuning. Chapter 5 presents MegaScale, the production-grade system for 10K-GPU training, covering its full-stack optimizations, fault tolerance mechanisms, and production experience. This chapter highlights MegaScale’s key contributions: the full-stack algorithm-system co-design approach achieving 55.2% MFU on 12,288 GPUs, the comprehensive communication overlapping techniques, the robust fault tolerance framework supporting uninterrupted training across hundreds of failures, and the extensive observability tools for diagnosing complex issues at scale. Finally, Chapter 6 concludes the thesis with a comprehensive summary of

contributions from all three systems, discusses the implications of this research for the field, and outlines promising future research directions including scaling to hundred-thousand GPU clusters, supporting heterogeneous hardware, and improving automation and observability.

Chapter 2

Background and Related Work

To understand the design of the distributed LLM training systems, we first introduce the fundamental background of parallelism techniques in distributed deep learning, which are the building blocks of these systems. We then discuss the related work in this field, which lays the foundation for the development of the three systems we present in this thesis.

2.1 Fundamental Parallelism Techniques

In distributed deep learning, there are three fundamental parallelism paradigms that are widely used to scale the training of large models: data parallelism, tensor parallelism, and pipeline parallelism. These techniques address different bottlenecks of large-scale training, and can be combined together to achieve better scalability.

Data parallelism is the most traditional parallelism technique, which replicates the entire model across multiple workers, and partitions the training data across these workers. Each worker computes the gradients independently on its own data split, and then synchronizes the gradients across all workers using all-reduce communication operations. This approach is simple to implement, and scales well for small models, but it has a critical limitation: the entire model must fit into the memory of each worker, which makes it impossible to train very large models that exceed the memory of a single GPU. To address this limitation, ZeRO [22] further extends data parallelism by sharding the optimizer states, gradients, and parameters across workers, reducing the memory redundancy. ZeRO achieves this through three stages of optimization: ZeRO-1 partitions optimizer states only, ZeRO-2 partitions optimizer states and gradients, and ZeRO-3 partitions optimizer states, gradients, and parameters. This dramatically reduces the memory per worker while maintaining similar communication volume to traditional data parallelism. However, even with ZeRO, large-scale training still faces communication bottlenecks for very large models, as the parameter sync overhead grows with model size.

Tensor parallelism, also known as intra-operator parallelism, partitions the computation of individual operators across multiple workers. For example, in a matrix multiplication operation, the weight matrix can be partitioned along the column or row dimension, so that each worker only needs to store a part of the weights, and computes a part of the output. This approach reduces the memory requirement per worker, enabling the training of larger

models. However, it requires frequent communication between workers to synchronize the intermediate results, which can become a bottleneck when the workers are across different nodes with limited network bandwidth. The key challenge in tensor parallelism is minimizing the communication overhead while maximizing the parallelism. Modern approaches like Megatron-LM’s column and row parallelism for MLP layers and attention head partitioning achieve this by exploiting the transformer architecture’s specific structure.

Pipeline parallelism, also known as inter-operator parallelism, partitions the model layers across multiple workers, so that each worker is responsible for a subset of the layers. The training batch is split into multiple microbatches, which are pipelined through the layers across workers. This approach allows the workers to process different microbatches in parallel, overlapping the computation across layers. It requires less communication than tensor parallelism, as workers only need to communicate the intermediate activations between adjacent layers. However, it introduces pipeline bubbles, which are the idle time of workers when they are waiting for the previous microbatch to arrive, reducing the overall efficiency. Various techniques have been proposed to reduce pipeline bubbles, including interleaved schedules [19], asynchronous execution [11], and micro-batch size optimization.

These three parallelism techniques can be combined together to form 3D parallelism, which is widely used in modern LLM training systems. By combining data parallelism, tensor parallelism, and pipeline parallelism, systems can scale the training to thousands of GPUs, balancing the memory and communication requirements. The data parallel dimension handles the overall batch size, the tensor parallel dimension handles the within-layer computation, and the pipeline parallel dimension handles the between-layer computation. Finding the optimal configuration for a given model and cluster requires careful tuning of the parallelism degrees in each dimension.

2.2 Related Work

Early work on distributed deep learning focused on data parallelism, with systems like Horovod [24] and PyTorch DDP [16] optimizing the gradient synchronization process. Horovod introduced ring-allreduce for efficient gradient communication, which became a standard approach for data parallel training. PyTorch DDP built upon these ideas, integrating tightly with PyTorch’s autograd system to provide easy-to-use distributed training. These systems are efficient for small models, but cannot handle the large models that emerged later. To address the memory bottleneck, model parallelism techniques were proposed. GPipe [11] pioneered the pipeline parallelism approach, which enables training of billion-parameter models by pipelining microbatches across layers. It demonstrated near-linear scaling for homogeneous models but required homogeneous layer structures. Mesh-TensorFlow [25] introduced the tensor parallelism approach, allowing users to manually specify the partitioning of operators across devices. These works laid the foundation for the distributed training of large models.

As the models grew larger, researchers started to explore automated parallelization systems, which could reduce the engineering burden of manual parallelism design. FlexFlow [12] proposed a randomized search approach to find the optimal parallelization plan, but it could not scale to large graphs due to the exponential search space. Tofu [31] devel-

oped a dynamic programming algorithm for intra-operator parallelism, but it was limited to single-node execution and could not handle cross-node parallelism. Varuna [1] focused on automating pipeline and data parallelism for low-bandwidth clusters, but it did not support tensor parallelism and required significant manual specification. DeepSpeed [23] provided system optimizations for large-scale training including ZeRO stages and 3D parallelism, but still required manual configuration of parallelism parameters. Alpa, which we will discuss in detail later, unified these approaches into a hierarchical framework, enabling automatic parallelization across multiple nodes and different parallelism dimensions with minimal user specification.

Most recently, as the training scale increased to tens of thousands of GPUs, researchers started to focus on the practical challenges of production training. Fault tolerance and straggler mitigation became critical issues, as failures became common at this scale. Systems like Pingmesh [9] and NetBouncer [27] were developed to diagnose network issues in data centers, providing the diagnostic infrastructure necessary for large-scale deployments. Hostping [17] focused on intra-host network bottlenecks in RDMA servers, identifying issues that are invisible to traditional network monitoring. These works provided the diagnostic tools that enabled large-scale production training, which are integrated into systems like MegaScale to enable stable training at scale.

Beyond the specific systems discussed in this thesis, there has been significant research on attention optimization and memory reduction techniques. FlashAttention [5, 6] developed IO-aware attention algorithms that reduce memory usage from quadratic to linear in sequence length while maintaining exact attention computation. These techniques have become essential building blocks for training long-context models. Gradient checkpointing [3] reduces memory requirements by recomputing activations during backpropagation, trading compute for memory. Mixed precision training [18] uses fp16 for forward and backward passes while maintaining fp32 for weight updates, significantly reducing memory and improving throughput on Tensor Cores.

The survey paper by Zeng et al. [33] provides a comprehensive overview of distributed training techniques, categorizing approaches by parallelism strategy, communication optimization, and fault tolerance mechanisms. Recent benchmark efforts aim to standardize evaluation of distributed training systems, enabling more meaningful comparisons across different approaches.

Chapter 3

Megatron-LM: Manual Tensor Parallelism for Large Model Training

Megatron-LM, proposed in 2019, is the pioneering system that enabled the training of billion-parameter transformer models through manual tensor parallelism. Before Megatron-LM, existing model parallelism systems either required custom compilers or could not achieve high efficiency for transformer models. Megatron-LM addressed these limitations by proposing a simple yet efficient intra-layer model parallelism approach, which only required a few modifications to the native PyTorch implementation, without any custom compiler or library changes. This system demonstrated that it is possible to train transformer models with up to 8.3 billion parameters on 512 GPUs, achieving 76

3.1 Motivation

The rapid growth of large language models has led to an unprecedented demand for computational resources. As model sizes escalated from millions to hundreds of billions of parameters, the training infrastructure faced critical bottlenecks that threatened to halt progress in the field. A single NVIDIA V100 GPU, with its 32GB HBM memory, cannot even hold the parameters of a 10 billion parameter model in fp16 precision, let alone perform the forward and backward computations required for training. This fundamental memory constraint rendered traditional single-GPU training approaches obsolete for the largest models.

Prior to Megatron-LM, existing distributed training systems suffered from two major limitations that prevented efficient large-scale model training. First, systems requiring custom compilers, such as those based on XLA [28], introduced significant complexity and compilation overhead that hindered rapid experimentation and iteration. Researchers and engineers were forced to navigate intricate compiler toolchains, debug opaque optimization passes, and accept prolonged compilation times that slowed the development cycle considerably. Second, early model parallelism approaches like those in GPipe [11] achieved efficiency primarily through pipeline parallelism but treated tensor parallelism as a secondary concern, resulting in suboptimal GPU utilization when scaling to large clusters.

These limitations became increasingly problematic as the field moved toward models with tens of billions of parameters. The engineering burden of manually designing parallelism

strategies for each new model architecture consumed valuable research resources and created bottlenecks that only highly specialized teams could overcome. This situation threatened to centralize large-scale LLM training among a few privileged organizations with dedicated systems teams, thereby slowing down the overall pace of innovation in the field.

The core motivation behind Megatron-LM was to demonstrate that efficient tensor parallelism could be achieved through simple modifications to standard deep learning frameworks, without sacrificing the flexibility and ease-of-use that researchers expected from modern tooling. By exploiting the specific architectural characteristics of transformer models, Megatron-LM showed that communication-efficient tensor parallelism could be implemented with minimal changes to existing PyTorch code, opening the door to large-scale training for a broader research community.

As shown in Figure 3.1, the system enables the training of models that are orders of magnitude larger than previous systems, with significantly reduced training time. This breakthrough allowed researchers to explore the scaling laws of language models, demonstrating that larger models consistently achieve better performance, which drove the subsequent development of even larger models.

Contributions. The primary contributions of Megatron-LM are summarized as follows: (1) We propose a novel intra-layer tensor parallelism approach that exploits the inherent structure of transformer architectures, achieving communication efficiency through careful partitioning of MLP and attention computations. (2) We demonstrate that this approach can be implemented with minimal modifications to native PyTorch, requiring no custom compilers or specialized libraries. (3) We show that the system achieves 76% scaling efficiency when training an 8.3 billion parameter model on 512 GPUs, sustaining 39 TeraFLOPs per GPU. (4) We provide architectural insights for scaling transformer models, including the critical finding that pre-LN architecture stabilizes training for large models. (5) We open-source the implementation, democratizing access to large-scale LLM training for the research community.

3.2 Design of Tensor Parallelism for Transformers

The key insight of Megatron-LM is to exploit the inherent structure of transformer layers to design the tensor parallelism strategy. A standard transformer layer consists of two main components: a self-attention block and a feed-forward network (MLP) block. Megatron-LM parallelizes these two blocks separately, minimizing the communication overhead while maximizing the computation parallelism.

For the MLP block, the standard computation consists of two linear layers with a GeLU activation in between. Megatron-LM partitions the first linear layer along the output dimension, so that each GPU only stores a part of the weight matrix. This allows each GPU to compute the corresponding part of the intermediate activation independently, without any communication, since the GeLU activation can be applied element-wise. The second linear layer then takes the partitioned intermediate activation as input, and computes the output. After the second linear layer, an all-reduce operation is performed across the GPUs to aggregate the results, which gives the final output of the MLP block. This approach only requires one all-reduce operation per MLP block, which minimizes the communication overhead.

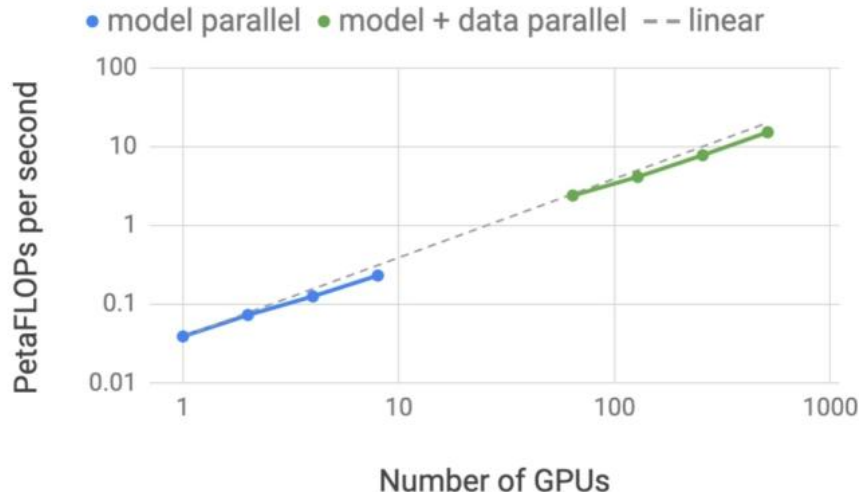


Figure 3.1: Scaling of language model training. The figure shows that Megatron-LM enables training of much larger models with significantly reduced training time compared to previous systems.

For the self-attention block, Megatron-LM partitions the attention heads across GPUs. Since each attention head works independently, partitioning the heads across GPUs allows each GPU to compute the attention computation for its own heads independently, without any communication. The output of the attention layer is then aggregated across GPUs using an all-reduce operation, similar to the MLP block. This approach also only requires one all-reduce operation per attention block, keeping the communication overhead low.

As shown in Figure 3.2, this design results in only four communication operations per transformer layer: two all-reduce operations in the forward pass, and two in the backward pass. Figure 3.3 illustrates this communication pattern in detail, showing how the minimal synchronization points allow computation to be overlapped with communication as much as possible. Moreover, this approach is orthogonal to pipeline parallelism, meaning that it can be combined with pipeline parallelism to further scale the model size.

Megatron-LM also combines this tensor parallelism with data parallelism to form a hybrid parallelism strategy. As shown in Figure 3.4, GPUs are grouped into model parallel groups, where tensor parallelism is applied within the group, and data parallelism is applied across the groups. This allows the system to scale to thousands of GPUs, while keeping the tensor parallelism communication within the node, where the bandwidth is higher.

3.3 Architectural Adjustments for BERT Models

During the development of Megatron-LM, the researchers found that the original BERT architecture could not scale to larger models, as the training would become unstable and the performance would degrade. They investigated this issue and found that the problem was caused by the placement of the layer normalization and residual connections. In the original BERT architecture, the layer normalization is applied after the residual connection, which leads to gradient instability as the model size grows. To address this issue, they rearranged

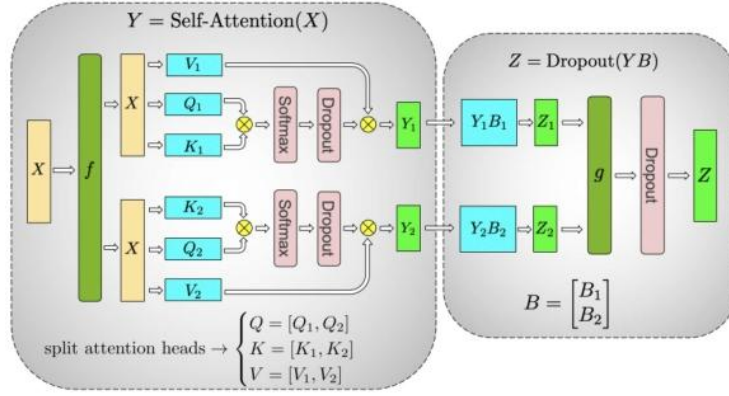
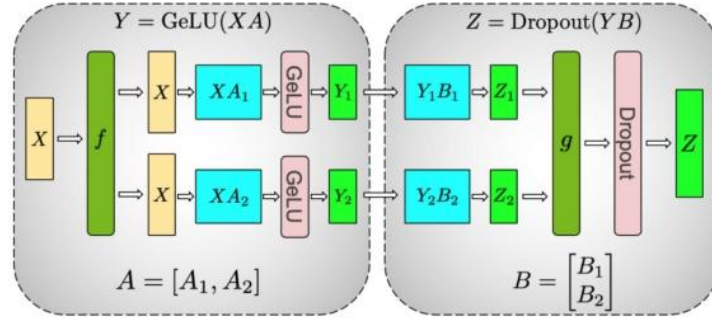


Figure 3.2: Parallel blocks of Transformer with Model Parallelism. The figure shows how the MLP and self-attention blocks are parallelized across GPUs, with the f and g operators handling the all-reduce communication.

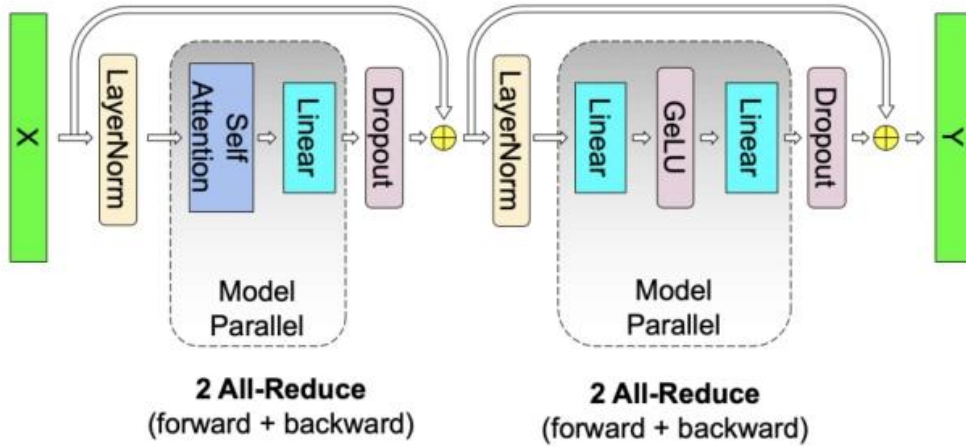


Figure 3.3: Communication operations in a transformer layer. The figure illustrates that there are only 4 total communication operations in the forward and backward pass of a single model parallel transformer layer, which minimizes the communication overhead.

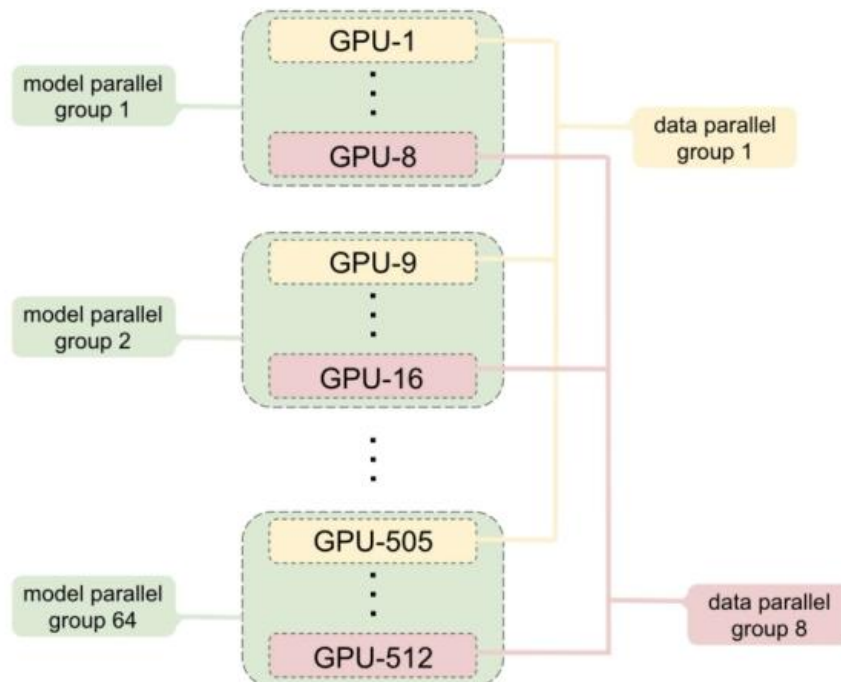


Figure 3.4: Grouping of GPUs for hybrid model and data parallelism. The figure shows the logical grouping of GPUs, where model parallel groups are within nodes, and data parallel groups span across nodes.

the order, applying the layer normalization before the residual connection, which stabilizes the training and allows the model to scale to larger sizes. As shown in Figure 3.5, this simple adjustment enabled the training of BERT models with up to 3.9 billion parameters, with monotonically improving performance as the model size increases.

3.4 Experimental Results

To evaluate the performance of Megatron-LM, the researchers conducted extensive experiments on a cluster of DGX-2H servers, with 512 V100 GPUs in total. They evaluated both the scalability of the system and the performance of the trained models.

3.4.1 Scalability Analysis

First, they evaluated the weak scaling performance of the system, scaling the model size along with the number of GPUs. The model configurations are shown in Table 3.1. They tested both tensor parallelism alone, and the hybrid tensor and data parallelism.

The results show that the system achieves excellent scaling efficiency. Figure 3.6 demonstrates that the system maintains high scaling efficiency even up to 512 GPUs. For the 8.3 billion parameter model on 512 GPUs, the system achieves 74

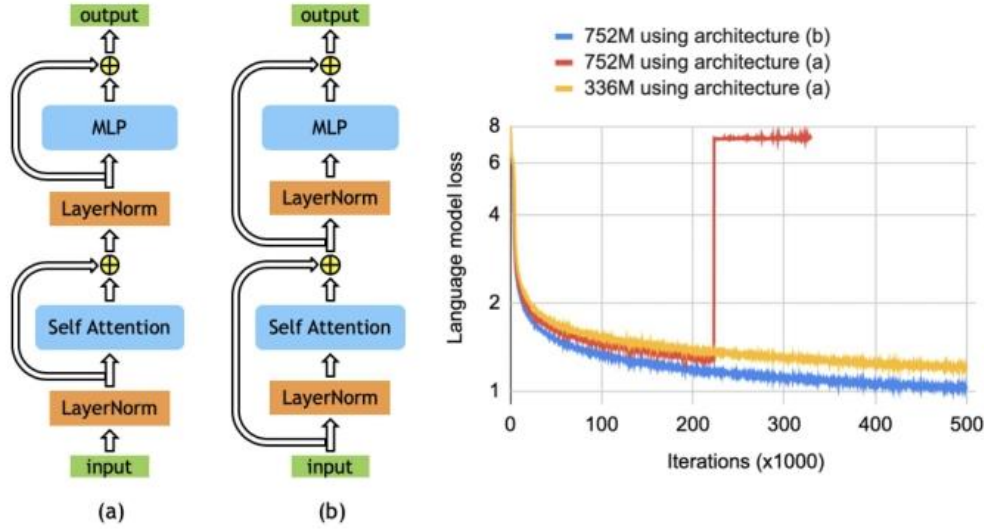


Figure 3.5: Training loss for BERT model using the original architecture and the rearranged architecture. The left part shows that the original architecture fails to scale, while the rearranged architecture enables stable training with lower loss.

Table 3.1: Parameters used for scaling studies. Hidden size per attention head is kept constant at 96.

Hidden Size	Attention heads	Layers	Parameters (B)	Model parallel GPUs	Model+data GPUs
1536	16	40	1.2	1	64
1920	20	54	2.5	2	128
2304	24	64	4.2	4	256
3072	32	72	8.3	8	512

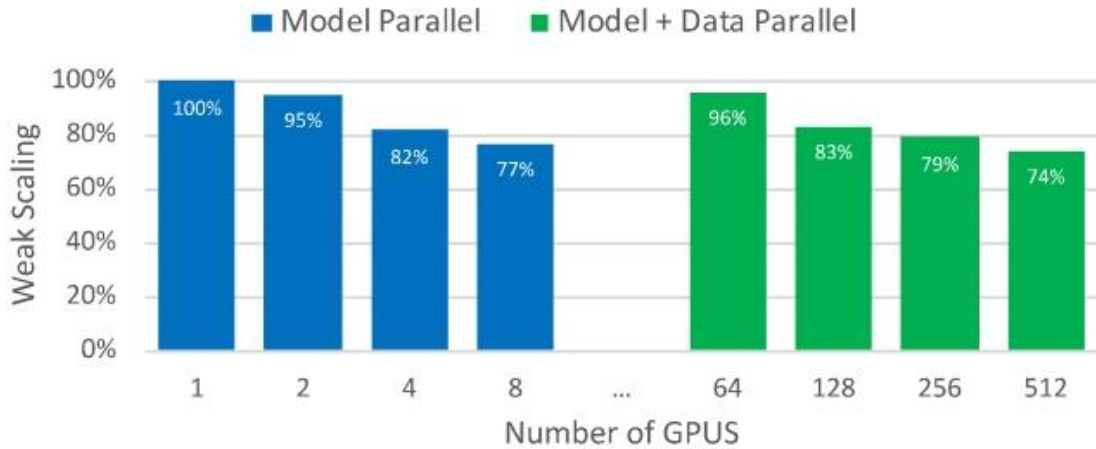


Figure 3.6: Model and model + data parallel weak scaling efficiency as a function of the number of GPUs. The system maintains high scaling efficiency even up to 512 GPUs.

3.4.2 Language Modeling Results with GPT-2

They also trained GPT-2 style models of different sizes, and evaluated their performance on language modeling tasks. The model configurations are shown in Table 3.2. They evaluated the models on WikiText103 perplexity and LAMBADA cloze accuracy, which are standard benchmarks for language models.

Table 3.2: Model configurations used for GPT-2.

Parameter Count	Layers	Hidden Size	Attn Heads	Hidden Size per Head	Total GPUs	Time per Epoch (days)
355M	24	1024	16	64	64	0.86
2.5B	54	1920	20	96	128	2.27
8.3B	72	3072	24	128	512	2.10

The results show that as the model size increases, the performance improves monotonically. As shown in Table 3.3, the 8.3B model achieves state-of-the-art results on both benchmarks: a perplexity of 10.81 on WikiText103, which is significantly better than the previous SOTA of 15.79, and an accuracy of 66.51

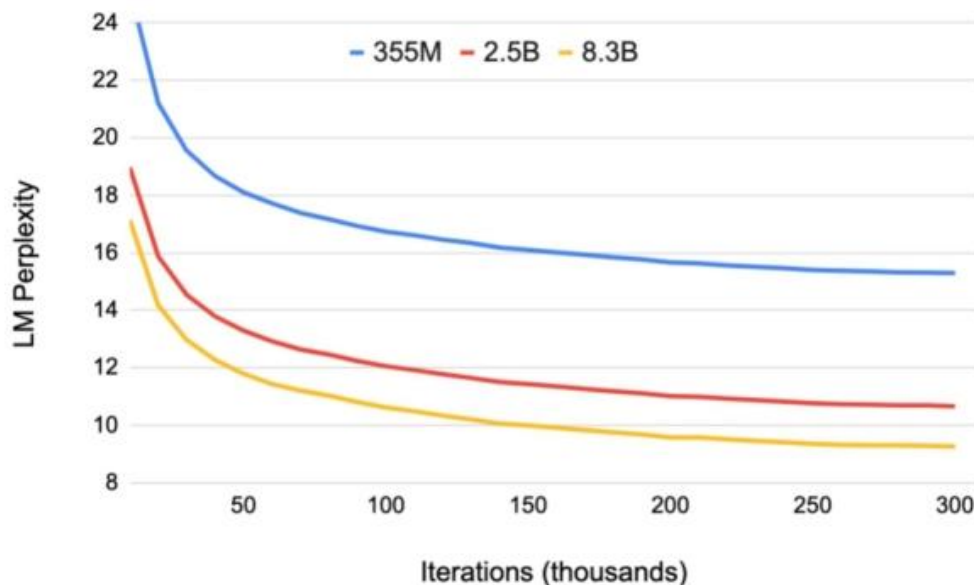


Figure 3.7: Validation set perplexity. Larger language models converge noticeably faster and converge to lower validation perplexities than their smaller counterparts.

3.4.3 BERT Results

For the BERT models, they trained models of up to 3.9 billion parameters, and evaluated their performance on downstream tasks. Table 3.4 shows that with the architectural ad-

Table 3.3: Zero-shot results on WikiText103 and LAMBADA.

Model	Wikitext103 Perplexity	LAMBADA Accuracy
355M	19.31	45.18%
2.5B	12.76	61.73%
8.3B	10.81	66.51%
Previous SOTA	15.79	63.24%

justment, the performance improves monotonically with the model size. The 3.9B model achieves state-of-the-art results on the RACE dataset, with 90.9

Table 3.4: Downstream task results for BERT models.

Model	MNLI	QQP	SQuAD 1.1	SQuAD 2.0	RACE
RoBERTa	90.2/90.2	92.2	94.6/88.9	89.4/86.5	83.2
ALBERT	90.8	92.2	94.8/89.3	90.2/87.4	86.5
XLNet	90.8/90.8	92.3	95.1/89.7	90.6/87.9	85.4
Megatron-336M	89.7/90.0	92.3	94.2/88.0	88.1/84.8	83.0
Megatron-1.3B	90.9/91.0	92.6	94.9/89.1	90.2/87.1	87.3
Megatron-3.9B	91.4/91.4	92.7	95.5/90.0	91.2/88.5	90.9

These results demonstrated that Megatron-LM could enable the training of large transformer models with high efficiency and performance, which was a major breakthrough in the field of distributed LLM training.

3.5 Summary

This chapter presented Megatron-LM, the pioneering system that enabled efficient training of billion-parameter transformer models through manual tensor parallelism. We began by motivating the need for tensor parallelism, showing that the memory constraints of single GPUs made distributed training essential for scaling to large models. We then detailed the design of the tensor parallelism strategy, which exploits the architectural structure of transformer layers to minimize communication overhead while maximizing parallelism.

The key innovation of Megatron-LM lies in its communication-efficient design. By partitioning the MLP block along the output dimension and the attention heads across GPUs, the system achieves only four all-reduce operations per transformer layer in both forward and backward passes. This minimal communication pattern allows computation to overlap effectively with communication, achieving high GPU utilization even when tensor parallelism spans multiple nodes.

We also highlighted the critical architectural adjustment for scaling BERT models, where moving from post-LN to pre-LN architecture stabilizes training for large models. This finding has become a standard practice in modern LLM training and influenced subsequent transformer designs.

The experimental results validated the effectiveness of the approach. On 512 V100 GPUs, Megatron-LM achieved 74% scaling efficiency when training an 8.3 billion parameter model, sustaining 39 TeraFLOPs per GPU (30% of peak FLOPs). The trained models achieved state-of-the-art results on language modeling benchmarks, with GPT-2 8.3B achieving 10.81 perplexity on WikiText103 and 66.51% accuracy on LAMBADA, significantly surpassing previous best results.

The impact of Megatron-LM extends beyond these immediate results. By demonstrating that efficient tensor parallelism could be achieved through simple modifications to PyTorch, Megatron-LM democratized access to large-scale LLM training and laid the foundation for all subsequent distributed LLM training systems. Many modern systems, including those discussed in the following chapters, build directly upon the tensor parallelism techniques pioneered by Megatron-LM.

Chapter 4

Alpa: Automated Hierarchical Parallelism for Generalizable Training

While Megatron-LM achieved remarkable performance for transformer models, it required manual tuning of the parallelism parameters, and could not generalize to other model architectures. As researchers started to train more diverse models, the burden of manual parallelism design became a major bottleneck. To address this limitation, Alpa was proposed in 2022, which is an automated parallelization system that can automatically generate efficient parallel execution plans for general models, without any manual tuning. Alpa introduces a novel hierarchical view of parallelism, decomposing the problem into inter-operator and intra-operator parallelism, which allows it to unify all existing parallelism techniques and automatically find the optimal plan.

4.1 Motivation

While Megatron-LM demonstrated impressive scaling results for transformer models, its manual parallelism approach introduced significant practical challenges that limited its broader applicability. The tensor parallelism degree, pipeline parallelism configuration, and data parallelism strategy all required careful hand-tuning based on the specific model architecture and cluster topology. This manual process demanded deep expertise in distributed systems and considerable engineering effort for each new model variant.

As the deep learning field matured, researchers began exploring more diverse architectures beyond standard transformers. Mixture-of-Experts (MoE) models like GShard [15] featured heterogeneous computation patterns where different experts required different amounts of computation. Vision transformers [7] introduced new attention mechanisms and patch embeddings. Large recommender models [20] contained embedding tables and dense layers with fundamentally different parallelization requirements. For each of these architectures, the manual parallelism design process had to be repeated from scratch, consuming weeks or months of engineering effort.

The heterogeneity of hardware configurations further complicated the manual approach. Different GPU clusters featured varying inter-node bandwidths, different numbers of GPUs per node, and diverse network topologies. A parallelism strategy optimized for one cluster

often performed poorly on another, requiring re-tuning for each deployment environment. This lack of portability meant that research innovations in model architecture could not be easily transferred to production systems or different research clusters.

The root cause of these limitations was the fragmented treatment of parallelism techniques. Prior systems like Megatron-LM focused on tensor parallelism, GPipe [11] focused on pipeline parallelism, and ZeRO [22] focused on data parallelism, treating each in isolation. However, the optimal parallelization strategy often requires combining multiple parallelism dimensions simultaneously. Finding such combinations manually is challenging because the search space grows exponentially with the number of parallelism dimensions and cluster resources.

The core insight behind Alpa is that parallelism techniques can be naturally organized into a hierarchy based on their communication requirements and scalability properties. Intra-operator parallelism techniques like tensor parallelism and ZeRO require high-bandwidth communication within a group of devices and are suitable for partitioning computation within a single stage. Inter-operator parallelism techniques like pipeline parallelism require less communication and are suitable for partitioning computation across stages. By recognizing this hierarchy, Alpa can automatically navigate the large search space and find optimal configurations that combine multiple parallelism dimensions.

Contributions. The primary contributions of Alpa are summarized as follows: (1) We propose a novel hierarchical view of parallelism that unifies intra-operator and inter-operator parallelism, enabling systematic exploration of the combined parallelization space. (2) We design a two-level compilation approach with intra-operator and inter-operator passes, where each pass uses efficient algorithms (ILP and DP respectively) to find optimal configurations. (3) We implement cross-mesh resharding optimization that enables efficient communication across device meshes with different shapes, reducing communication overhead significantly. (4) We demonstrate that Alpa matches or exceeds hand-tuned systems like Megatron-LM for transformers while automatically handling heterogeneous models like Wide-ResNet and MoE models that lack manual parallelism solutions. (5) We make Alpa publicly available, providing the research community with automated access to efficient distributed training.

4.2 Hierarchical View of Parallelism

The key insight of Alpa is to re-categorize the existing parallelism techniques into two orthogonal categories: intra-operator parallelism and inter-operator parallelism. As shown in Figure 4.1, intra-operator parallelism refers to the techniques that partition the computation of individual operators across devices, such as data parallelism, tensor parallelism, and ZeRO. These techniques require frequent communication between devices, but they have high device utilization, as they can parallelize the computation within an operator. Inter-operator parallelism, on the other hand, refers to the techniques that partition the operators of the model across devices, such as pipeline parallelism. These techniques have less communication overhead, as they only communicate between adjacent stages, but they introduce pipeline bubbles, which reduce the device utilization.

This categorization allows Alpa to map the parallelism techniques to the cluster hierarchy. Intra-operator parallelism requires high bandwidth communication, so it is applied within a

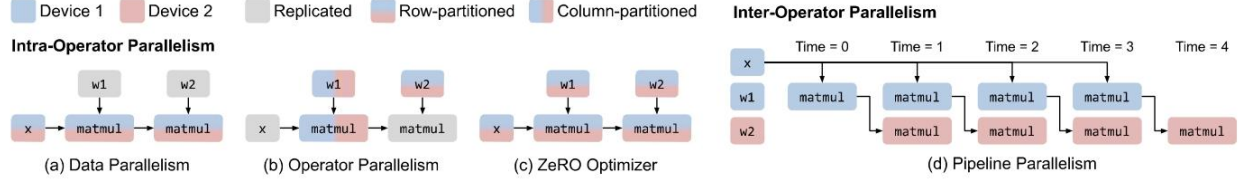


Figure 4.1: Parallelism techniques and their properties. The figure categorizes the existing parallelism techniques into intra-operator and inter-operator parallelism, showing their different communication and utilization properties.

device mesh, which is a group of devices that are connected with high bandwidth, such as the GPUs within a single node. Inter-operator parallelism, which requires less communication, is applied across the device meshes, which are connected with lower bandwidth across nodes. This hierarchical design allows Alpha to maximize the utilization of the cluster bandwidth, while minimizing the communication overhead.

As shown in Figure 4.2, this hierarchical approach enables a much larger space of parallelization plans than previous approaches. Previous approaches either only supported manual plans, or only supported one type of parallelism. Alpha’s approach unifies all of them, enabling the combination of inter- and intra-operator parallelism, which can adapt to different models and cluster environments.

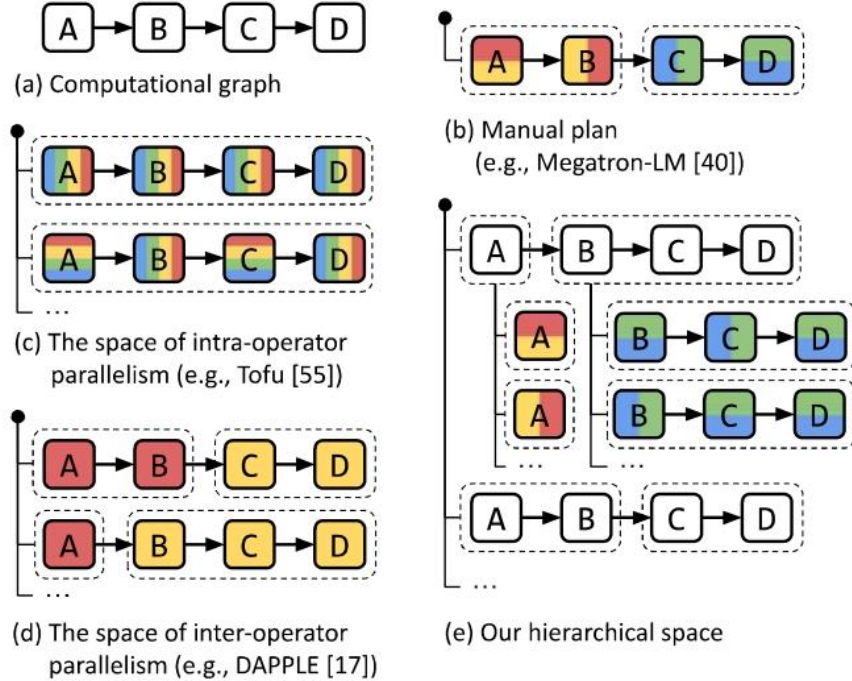


Figure 4.2: Generation of parallelization plans for a computational graph. The figure shows how Alpha’s hierarchical approach combines inter- and intra-operator parallelism, enabling a much larger plan space than previous approaches.

4.3 Compiler Passes

To automatically find the optimal parallelization plan, Alpa implements two compiler passes: the intra-operator pass and the inter-operator pass, which work hierarchically to optimize the plan.

4.3.1 Intra-Operator Parallelism Pass

The intra-operator pass optimizes the parallelization plan for a single stage (subgraph of the model) running on a device mesh. It formulates the problem as an integer linear programming (ILP) problem, which finds the optimal sharding strategy for each operator to minimize the total execution cost. The ILP problem considers both the computation cost and the communication cost, including the resharding cost between operators. It can automatically find the optimal sharding strategy, which unifies data parallelism, tensor parallelism, and ZeRO, without any manual specification. This pass can handle graphs with tens of thousands of operators, by merging trivial operators to reduce the problem size.

4.3.2 Inter-Operator Parallelism Pass

The inter-operator pass optimizes the partitioning of the model into stages, and the partitioning of the cluster into device meshes. It uses a dynamic programming (DP) algorithm to find the optimal way to split the model into stages, and assign each stage to a device mesh. The DP algorithm queries the intra-operator pass to get the execution cost of each stage-mesh pair, and finds the plan that minimizes the total pipeline latency. This approach automatically balances the computation across stages, and assigns the optimal device mesh size for each stage, based on its computation and memory requirements.

As shown in Figure 4.3, the two passes work together to generate the hierarchical parallelization plan. The inter-op pass first splits the model into stages, then the intra-op pass optimizes each stage, and the cost is fed back to the inter-op pass to find the optimal plan.

4.4 Runtime Orchestration

After the plan is generated, Alpa’s runtime orchestrates the execution across the cluster. For each stage, it compiles the stage into an SPMD executable, which runs on the device mesh using the intra-operator parallelism plan. For the communication between stages, Alpa implements cross-mesh resharding, which handles the communication between different device meshes with different shapes. As shown in Figure 4.4, it optimizes this communication by using local all-gather to move the communication from slow inter-node links to fast intra-node links, which significantly improves the performance.

The runtime also generates the pipeline execution instructions, which orchestrate the execution of the microbatches across the stages, enabling the pipelined execution. As shown in Figure 4.5, the pipeline execution significantly reduces the latency of the training, by overlapping the computation across stages.

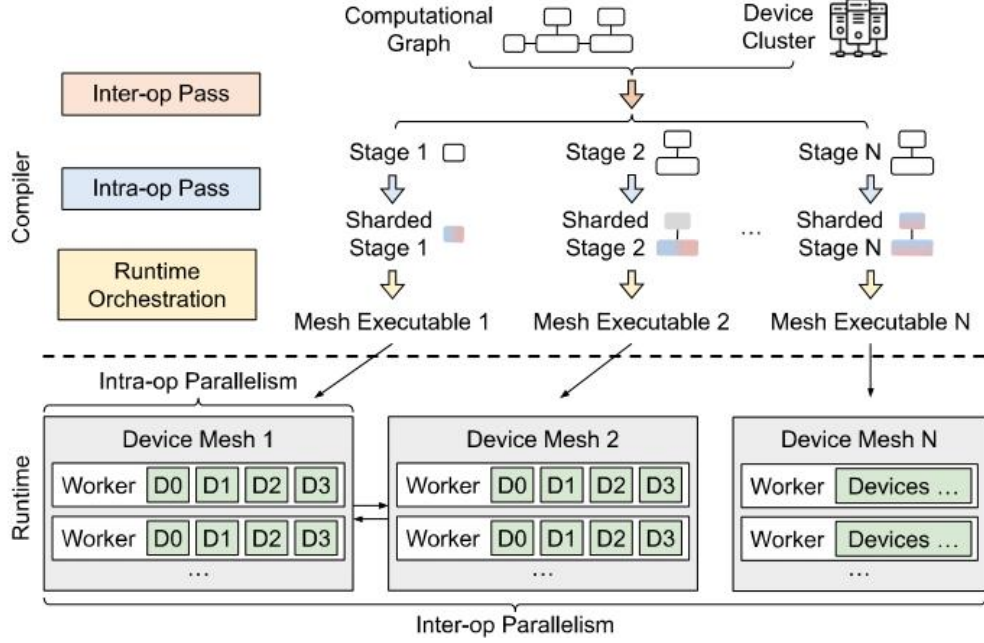


Figure 4.3: Compiler passes and runtime architecture. The figure shows the workflow of the compiler, which first runs the inter-op pass to split the model into stages, then runs the intra-op pass to optimize each stage, and then the runtime orchestrates the execution.

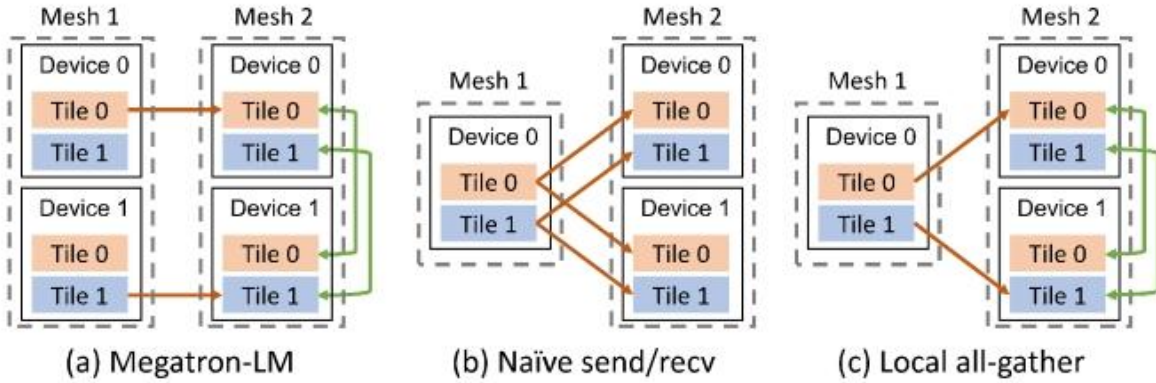


Figure 4.4: Cross-mesh resharding optimization. The figure shows how Alpa optimizes the cross-mesh communication by using local all-gather to reduce the inter-node communication overhead.

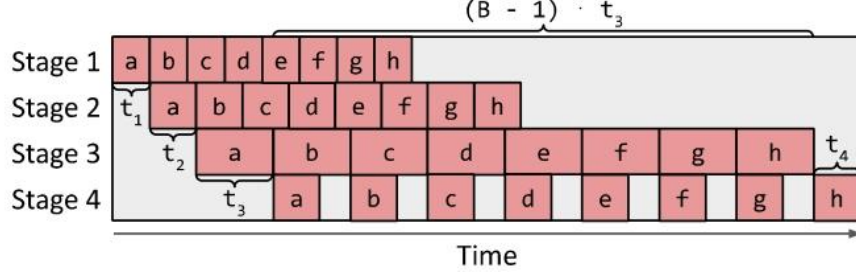


Figure 4.5: Pipeline execution latency.

4.5 Experimental Results

Alpa is evaluated on a cluster of 8 AWS p3.16xlarge instances, with 64 GPUs in total. The evaluation covers three types of models: GPT-3, GShard MoE, and Wide-ResNet, covering both homogeneous and heterogeneous models.

4.5.1 End-to-End Performance

The end-to-end evaluation shows that Alpa can match or even outperform the hand-tuned baselines. As shown in Figure 4.6, Alpa outperforms the baselines on all models, especially on MoE and Wide-ResNet. For GPT-3 models, Alpa achieves comparable performance to Megatron-LM, even though Alpa’s plan is automatically generated. For GShard MoE models, Alpa significantly outperforms DeepSpeed, achieving 3.5x speedup on 2 nodes and 9.7x speedup on 4 nodes. This is because DeepSpeed only uses intra-operator parallelism, which cannot scale across nodes, while Alpa combines inter- and intra-operator parallelism, enabling efficient scaling. For Wide-ResNet, which is a heterogeneous model with no manual parallelism plan, Alpa achieves 80

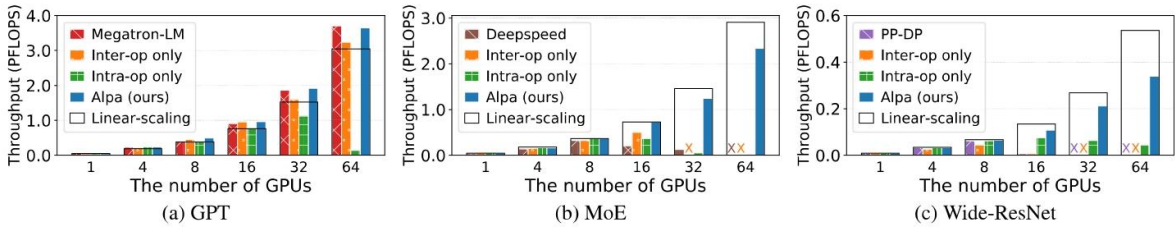


Figure 4.6: End-to-end evaluation results. The figure shows that Alpa outperforms the baselines on all models, especially on MoE and Wide-ResNet.

To further evaluate the cross-mesh performance, the researchers tested the performance of the cross-mesh resharding optimization. As shown in Figure 4.7, the optimization significantly reduces the communication time, especially for large tensors, which enables efficient scaling across nodes.

For the Wide-ResNet model, which is a heterogeneous model, Alpa automatically generates the parallelization plan, as shown in Figure 4.8. The plan partitions the model into

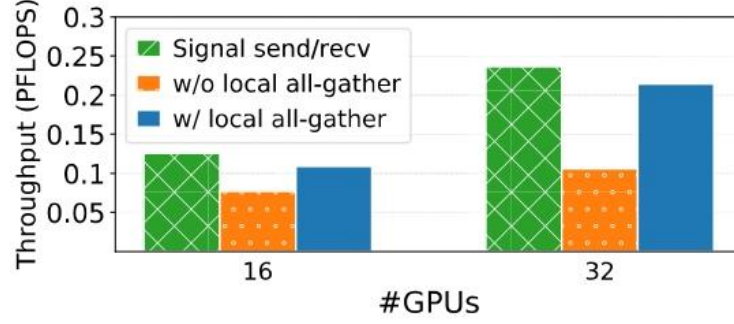


Figure 4.7: Cross-mesh resharding performance. The figure shows that the optimized resharding approach significantly outperforms the naive approach, reducing the communication time.

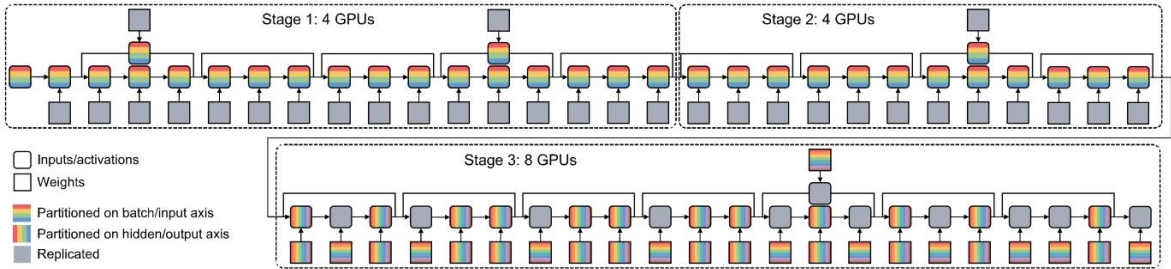


Figure 4.8: Parallelization plan for Wide-ResNet. The figure shows the automatically generated plan, which partitions the model into stages and assigns different mesh sizes to each stage.

stages, and assigns different device mesh sizes to each stage, balancing the computation across the stages, which enables efficient scaling.

4.5.2 Ablation Studies

The ablation studies show that both the intra-operator and inter-operator passes contribute to the performance. As shown in Figure 4.9, the ILP-based intra-operator pass significantly improves the performance over baseline approaches like ZeRO, by optimizing the sharding strategy to minimize the communication overhead. Figure 4.10 demonstrates that the DP-based inter-operator pass improves the performance over rule-based partitioning, especially for heterogeneous models, by automatically balancing the stages and assigning the optimal device mesh size.

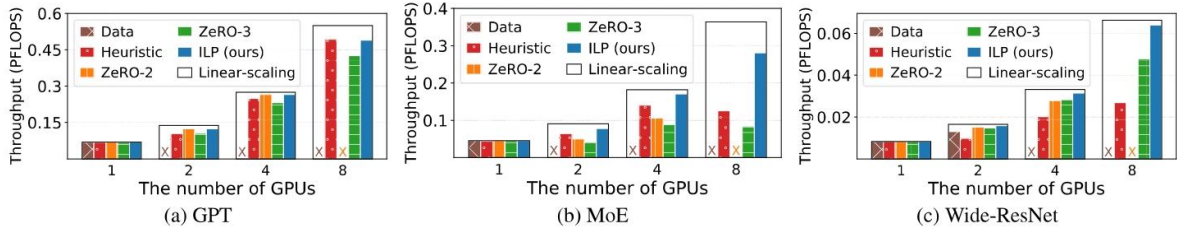


Figure 4.9: Intra-operator parallelism ablation, showing that the ILP-based approach outperforms other baselines.

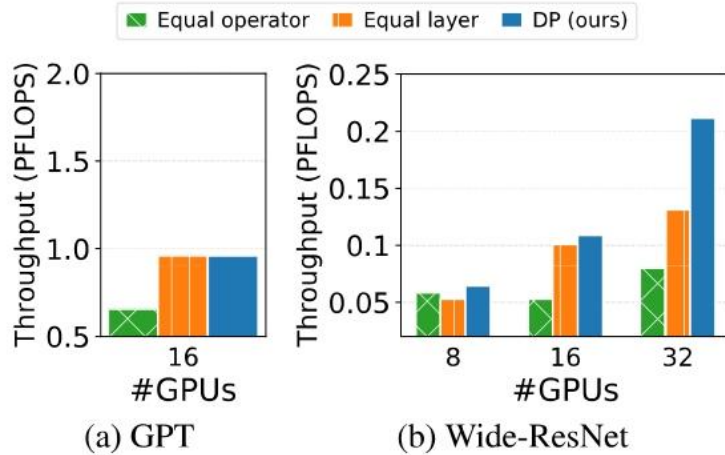


Figure 4.10: Inter-operator parallelism ablation, showing that the DP-based approach outperforms rule-based partitioning.

Additionally, the researchers evaluated the compilation time of Alpa’s compiler. As shown in Figure 4.11, the compiler can handle large models efficiently, with the compilation time increasing linearly with the number of operators, which means it can scale to very large models.

These results demonstrate that Alpa can automate the parallelization process, achieving performance comparable to hand-tuned systems, while generalizing to arbitrary models and

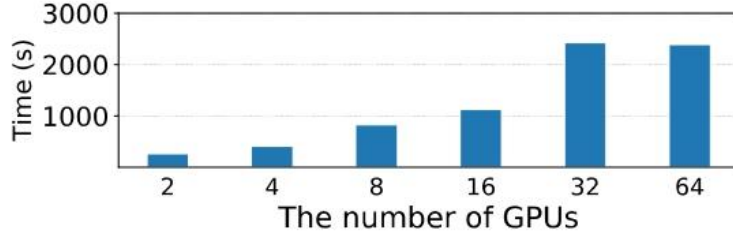


Figure 4.11: Compilation time of Alpa’s compiler. The compilation time scales linearly with the number of operators, enabling efficient handling of large models.

cluster environments. This democratizes the distributed training of large models, allowing researchers to train new models without the burden of manual parallelism design.

4.6 Summary

This chapter presented Alpa, an automated parallelization system that unifies inter-operator and intra-operator parallelism through a hierarchical compilation approach. We began by motivating the need for automation, showing that manual parallelism design becomes prohibitively expensive as model architectures diversify and hardware configurations proliferate.

The key innovation of Alpa lies in its hierarchical decomposition of the parallelization problem. By recognizing that parallelism techniques naturally group into intra-operator and inter-operator categories based on their communication requirements, Alpa enables systematic exploration of the combined parallelization space. The two-level compilation approach, with the intra-operator ILP pass and inter-operator DP pass, efficiently navigates this large search space to find optimal configurations.

We highlighted several important technical contributions. The ILP-based intra-operator pass unifies data parallelism, tensor parallelism, and ZeRO into a single formulation, automatically finding the optimal sharding strategy for each operator. The DP-based inter-operator pass automatically balances computation across stages and determines the appropriate device mesh size for each stage. The cross-mesh resharding optimization reduces communication overhead when scaling across multiple device meshes.

The experimental results validated Alpa’s effectiveness across diverse model architectures. For GPT-3, Alpa matched Megatron-LM’s hand-tuned performance. For GShard MoE, Alpa achieved up to 9.7x speedup over DeepSpeed by properly combining inter- and intra-operator parallelism. For Wide-ResNet, where no manual parallelism plan exists, Alpa achieved 80% scaling efficiency up to 32 GPUs, demonstrating automatic adaptation to heterogeneous computation patterns.

Alpa represents a significant step toward democratizing distributed LLM training. By eliminating the need for manual parallelism tuning, it enables researchers to focus on model innovation rather than system optimization. This automation is particularly valuable for the research community, where model architectures evolve rapidly and engineering resources are limited. The techniques developed in Alpa have influenced subsequent systems, particularly in the area of automated parallelization and hierarchical optimization.

Chapter 5

MegaScale: Full-Stack Optimization for 10K-GPU Production Training

While Alpa enabled automated parallelization for general models, it was still focused on research experiments, and could not handle the practical challenges of production-grade training at the scale of tens of thousands of GPUs. As LLMs started to be deployed in production, the need for a robust, efficient, and stable training system became critical. MegaScale, proposed in 2024, is a production-grade system that addresses these challenges, enabling the training of LLMs on more than 10,000 GPUs. It uses a full-stack algorithm-system co-design approach, optimizing every component of the training system to achieve high efficiency, and implements a robust fault tolerance framework with in-depth observability to handle the failures and stragglers that are common at this scale.

5.1 Motivation

The progression from research prototypes to production deployment reveals fundamental gaps that cannot be bridged by incremental improvements. While Megatron-LM and Alpa demonstrated impressive capabilities within controlled research environments, neither system was designed to address the multifaceted challenges that emerge when training at the scale of tens of thousands of GPUs in production data centers. These challenges span algorithmic efficiency, system robustness, operational complexity, and diagnostic capabilities, creating a gap that demands a holistic approach.

At the scale of 10,000+ GPUs, failures transition from exceptional events to regular occurrences. Statistical analysis of large-scale deployments reveals that with a mean time between failures (MTBF) of approximately 10,000 hours per GPU, a 10,000-GPU cluster will experience multiple failures per day on average. These failures take many forms: GPU hardware failures, network link degradations, memory errors, software bugs, and rack-level power issues. A training system that cannot gracefully handle such failures would spend more time restarting than actually training, rendering large-scale deployment impractical.

Beyond failures, performance variability becomes a dominant concern at scale. Stragglers—nodes that execute slower than expected due to hardware variations, thermal throttling, or software contention—can dramatically reduce overall throughput. In a 10,000-GPU train-

ing run, if even 1% of nodes are stragglers, the entire job may be bottlenecked by these slow workers. Traditional approaches that rely on synchronous gradient updates become particularly vulnerable, as the slowest worker determines the iteration time for all others.

The communication patterns at this scale introduce additional bottlenecks that do not exist in smaller deployments. Collective communications like all-reduce and all-gather must traverse complex network topologies involving multiple switches and links. Load imbalance across these paths can cause congestion, leading to priority flow control (PFC) storms that degrade network performance for all traffic. The bisection bandwidth of the network becomes critical, and suboptimal communication schedules can waste substantial network capacity.

Energy consumption and operational costs become first-class concerns in production environments. A 10,000-GPU cluster can consume tens of megawatts of power, costing millions of dollars per month in electricity alone. Inefficiencies in the training system directly translate to increased operational costs and carbon footprint. Optimizing for both performance and efficiency is therefore essential for sustainable large-scale deployment.

The complexity of diagnosing performance issues at scale exceeds the capabilities of traditional monitoring tools. When a training job runs for weeks on thousands of GPUs, identifying the root cause of a sudden MFU degradation requires correlating information across hundreds of variables: network metrics, GPU utilization, memory bandwidth, cache behavior, and application-level statistics. Without purpose-built observability tools, troubleshooting becomes an exercise in frustration.

MegaScale addresses these challenges through full-stack co-design, treating the training system as an integrated whole rather than a collection of independent components. By optimizing algorithms, communication, operators, data pipeline, and network together, MegaScale achieves efficiency unattainable through point optimizations. By building robust fault tolerance and comprehensive observability from the ground up, MegaScale enables stable production training at scales previously thought impractical.

Contributions. The primary contributions of MegaScale are summarized as follows: (1) We present a full-stack algorithm-system co-design approach that optimizes every component of distributed LLM training, from algorithmic choices to network-level communication, achieving 55.2% MFU on 12,288 GPUs. (2) We develop comprehensive communication overlapping techniques that hide communication overhead behind computation across data parallelism, pipeline parallelism, and tensor parallelism, significantly improving GPU utilization. (3) We implement a robust fault tolerance framework with automatic failure detection, diagnosis, and recovery, enabling uninterrupted training across hundreds of failures. (4) We build extensive observability tools including CUDA event heat maps, distributed timeline traces, and 3D parallel visualization that enable diagnosis of issues impossible to detect with traditional monitoring. (5) We share production experience and insights from training LLMs at extreme scale, providing valuable guidance for practitioners deploying large-scale training systems.

5.2 3D Parallelism Foundation

MegaScale builds on top of the 3D parallelism framework, which combines data parallelism, tensor parallelism, and pipeline parallelism to scale the training. As shown in Figure 5.1 and

Figure 5.2, the system partitions the cluster into three dimensions, enabling efficient parallel execution of the training.

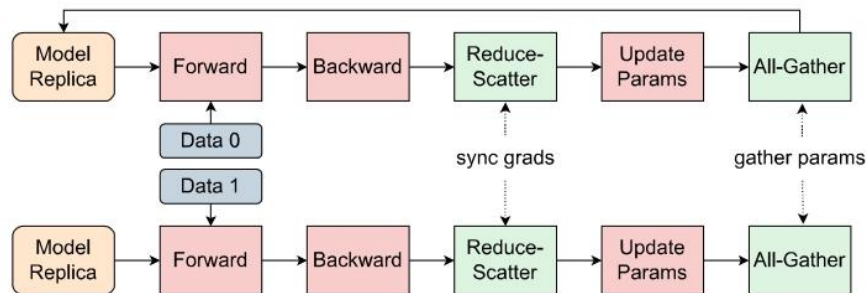


Figure 5.1: ZeRO data parallelism. The figure shows how the parameters are sharded across the data parallel dimension, reducing the memory requirement per worker.

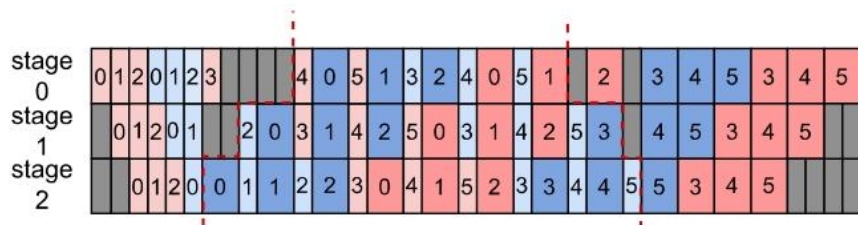


Figure 5.2: Pipeline parallelism. The figure shows how the model layers are partitioned across the pipeline parallel dimension, enabling pipelined execution of microbatches.

5.3 Full-Stack Efficiency Optimization

To achieve high training efficiency at scale, MegaScale optimizes every component of the training system, from algorithm design to communication, operators, data pipeline, and network.

5.3.1 Algorithmic Optimizations

First, MegaScale introduces several algorithmic optimizations to improve the training efficiency. It adopts the parallel transformer block, which parallelizes the computation of the attention and MLP blocks, reducing the computation time without affecting the model accuracy. It also uses sliding window attention, which reduces the computation complexity of the attention layer from $O(s^2)$ to $O(s \times w)$, where w is the window size, enabling faster training without losing the context information. Additionally, it uses the LAMB optimizer, which enables scaling the batch size by 4x without affecting the convergence, which reduces the pipeline bubbles and improves the efficiency.

5.3.2 Communication Overlapping

Communication is a major bottleneck of large-scale training. MegaScale systematically analyzes the dependencies of the communication operations in 3D parallelism, and designs techniques to overlap the communication with computation, hiding the communication overhead. For data parallelism, it overlaps the all-gather and reduce-scatter operations with the computation of the model chunks, pre-fetching the parameters before the computation. For pipeline parallelism, it decouples the send and receive operations, allowing them to overlap with the computation of the forward and backward passes. As shown in Figure 5.3, this significantly reduces the idle time of the workers.

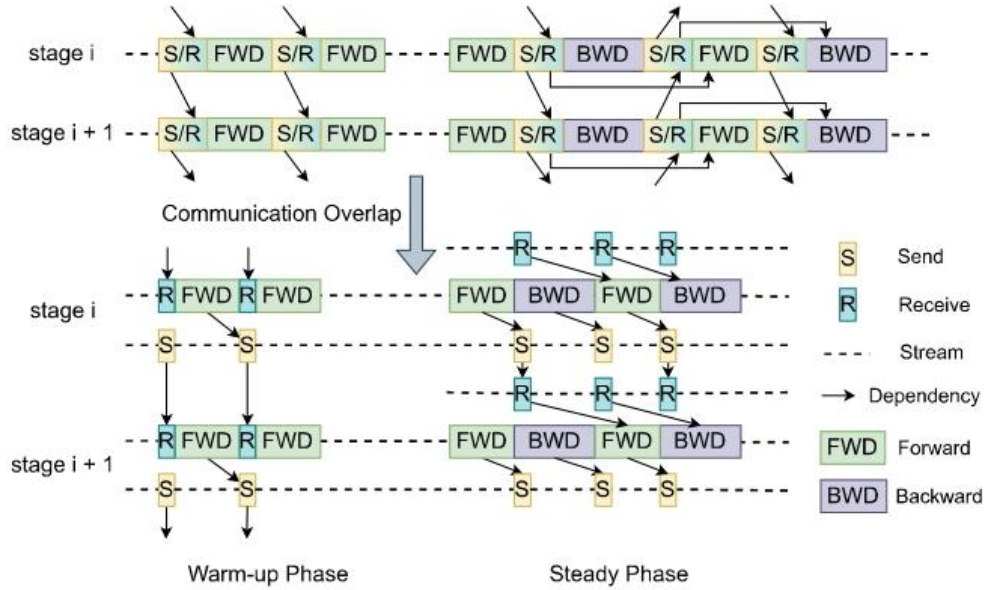


Figure 5.3: Overlapping communication in pipeline parallelism. The figure shows how the send and receive operations are overlapped with the forward and backward computation, hiding the communication overhead.

For tensor parallelism, it fuses the communication with the GEMM kernels, pipelining the execution to hide the communication overhead. As shown in Figure 5.4, these techniques significantly reduce the iteration time, improving the MFU (Model FLOPs Utilization) of the system.

5.3.3 Other System Optimizations

MegaScale also optimizes the operators, using FlashAttention-2 to accelerate the attention computation, and fusing the small kernels like LayerNorm and GeLU to reduce the kernel launch overhead. It also optimizes the data pipeline, using a tree-based data loader to eliminate redundant disk reads, and asynchronous preprocessing to overlap the data preprocessing with the training computation. Additionally, it optimizes the collective communication group initialization, replacing the slow TCPStore with Redis, and eliminating the global barriers, reducing the initialization time from 1000 seconds to under 30 seconds for 10K GPUs. It

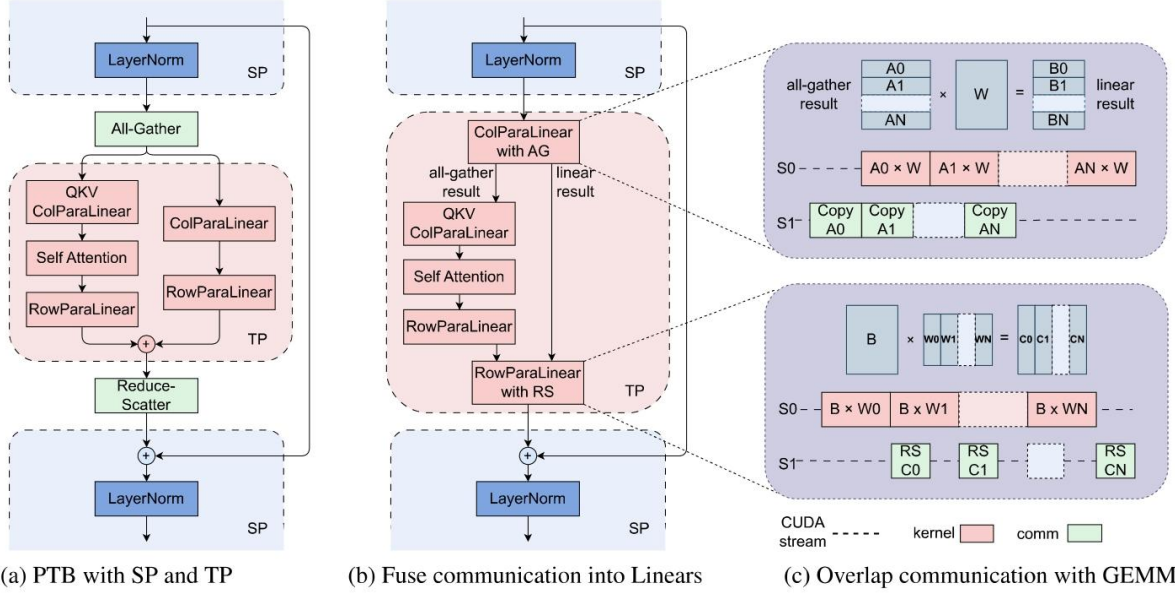


Figure 5.4: Overlapping communication in tensor parallelism. The figure shows how the communication is overlapped with the GEMM computation, hiding the communication overhead.

also tunes the network performance, optimizing the topology to reduce ECMP conflicts, and improving the congestion control to reduce the PFC storms, which improves the network stability and throughput.

5.4 Fault Tolerance and Observability

At the scale of 10K GPUs, failures and stragglers are common, so fault tolerance is critical. MegaScale implements a robust training framework that can automatically detect and recover from failures, with in-depth observability tools to diagnose the issues.

5.4.1 Robust Training Workflow

The robust training workflow, shown in Figure 5.5, automatically handles the failures. When a failure is detected, the system runs diagnostic tests to identify the faulty nodes, evicts them, and replenishes the cluster with healthy nodes, then resumes the training from the latest checkpoint. The checkpointing is optimized to be fast: the first stage dumps the state to host memory in seconds, then the background process writes it to HDFS asynchronously, minimizing the interruption to the training. The recovery is also optimized, using broadcast to share the checkpoint across workers, reducing the recovery time.

5.4.2 In-Depth Observability and Diagnosis Tools

To diagnose the complex issues at large scale, MegaScale implements a set of observability tools. The CUDA event monitor records the execution time of critical code segments, and

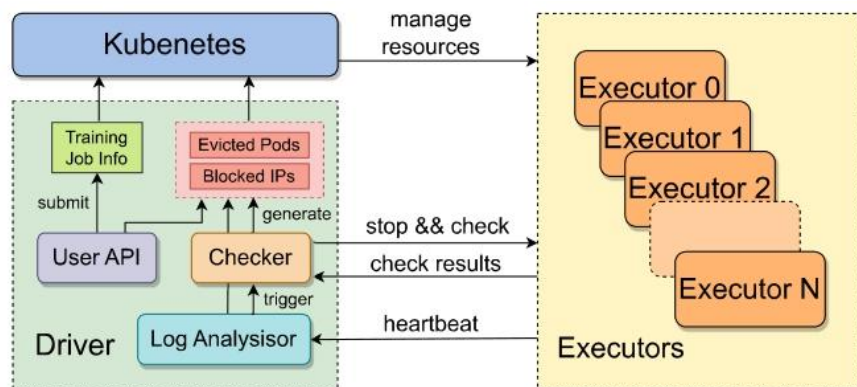


Figure 5.5: Robust training workflow. The figure shows how the system detects failures, runs diagnostics, evicts faulty nodes, and recovers the training from checkpoint.

generates heat maps to identify straggler nodes, which are the nodes that are slower than others. As shown in Figure 5.6, the heat map can clearly show the straggler nodes, which are the ones with longer execution time.

It also generates distributed timeline traces, which visualize the execution of the pipeline across nodes, helping to identify the pipeline bubbles and synchronization issues. As shown in Figure 5.7, the timeline trace can show the execution of each stage across nodes, enabling the diagnosis of pipeline issues.

The system also monitors the MFU of each node, to identify the nodes that have inconsistent performance. As shown in Figure 5.8, the inconsistent MFU indicates that there are straggler nodes that are affecting the overall performance.

The 3D parallel visualization tool helps to pinpoint the faulty nodes when the training hangs, by showing the data dependencies across the parallel dimensions. These tools enable the system to identify and fix the issues that cannot be detected by traditional monitoring tools.

5.5 Production Experience and Results

MegaScale is deployed in production, and has been used to train LLMs on more than 10,000 GPUs. The experimental results show that it achieves 55.2

The weak scaling results, shown in Figure 5.9, show that the system maintains high MFU even as the number of GPUs increases, up to 12,288 GPUs. This demonstrates that the system can scale efficiently to very large clusters.

The micro-batch loss results, shown in Figure 5.10, show that the training is stable, with the loss converging smoothly, even with the large batch size.

The production run shows that the system can maintain stable training for several weeks, recovering from over 100 failures automatically, and the model loss converges continuously. The system also fixed many practical issues, such as stragglers, network flapping, and MFU degradation, which are only encountered at large scale. As shown in Figure 5.11, the MFU remains stable throughout the entire training run, demonstrating the stability of the system.

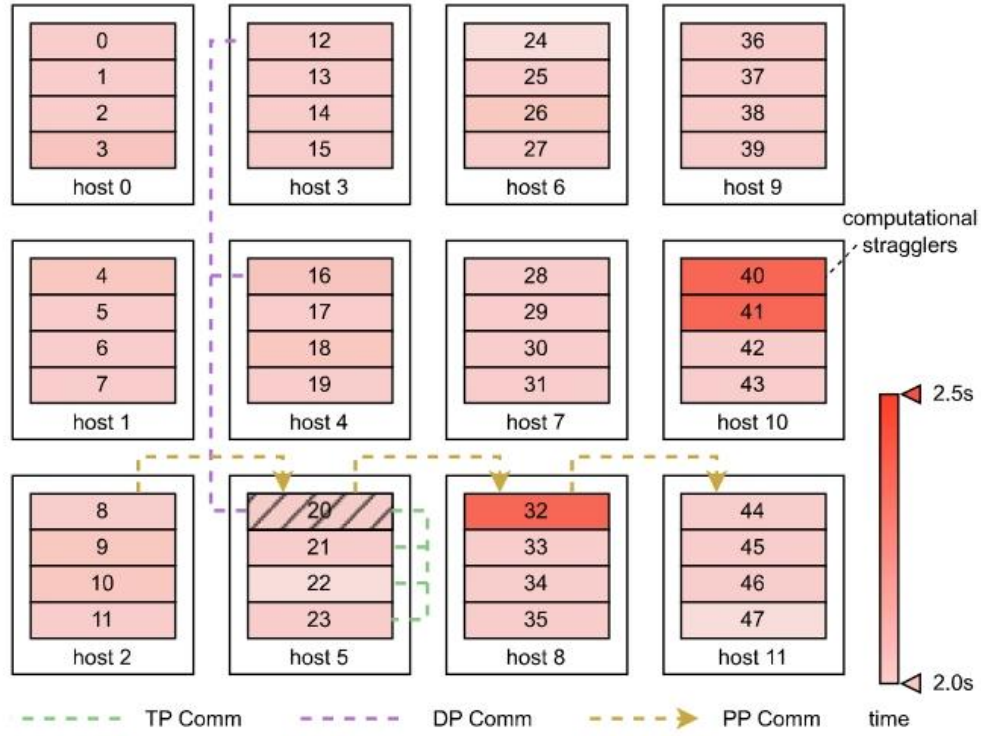


Figure 5.6: CUDA event heat map. The heat map visualizes the execution time of each node, enabling the identification of straggler nodes.

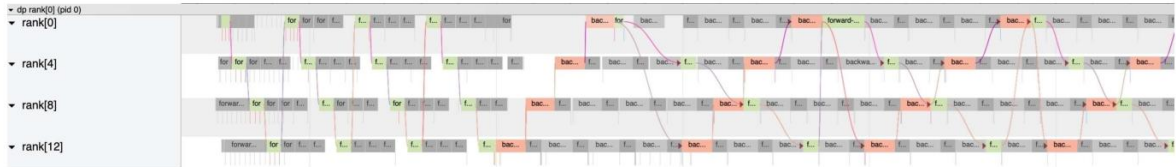


Figure 5.7: Distributed timeline trace. The trace visualizes the execution of the pipeline across nodes, helping to identify pipeline bubbles and synchronization issues.

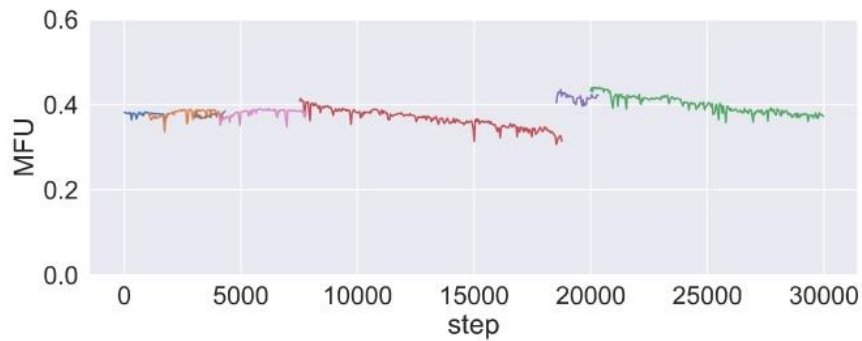


Figure 5.8: Inconsistent MFU across nodes. The figure shows that some nodes have lower MFU than others, indicating straggler issues.

Table 5.1: Strong-scaling training performance for the 175B model.

Batch Size	Method	GPUs	Iteration Time (s)	Throughput (tokens/s)	Training Time (days)	MFU	Speedup
768	Megatron-LM	256	40.0	39.3k	88.35	53.0%	1.0x
768	MegaScale	256	32.0	49.0k	70.86	65.3%	1.23x
6144	Megatron-LM	12288	8.57	1466.8k	2.37	41.2%	1.0x
6144	MegaScale	12288	6.34	1984.0k	1.75	55.2%	1.34x

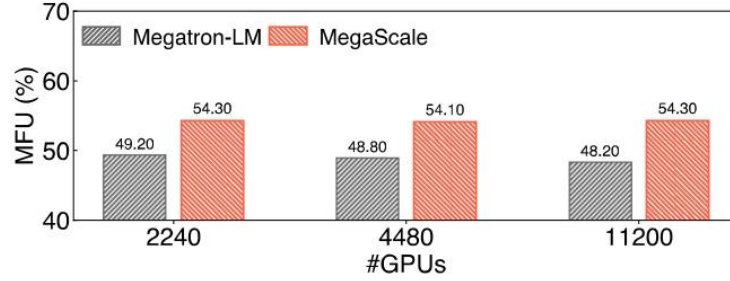
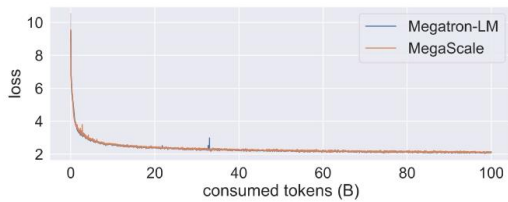
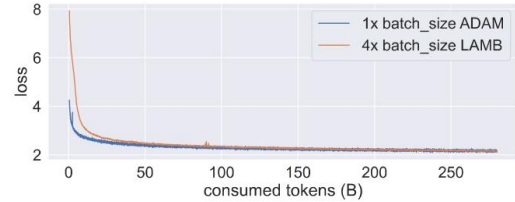


Figure 5.9: Weak scaling results. The figure shows that the system maintains high MFU as the number of GPUs increases, up to 12,288 GPUs.



(a) The training loss curve of MegaScale, which includes algorithm optimizations, in comparison with Megatron-LM.



(b) The training loss curve of ADAM optimizer and LAMB optimizer with four times of the batch size.

Figure 5.10: Micro-batch training loss. The loss converges smoothly, showing that the training is stable with the large batch size.

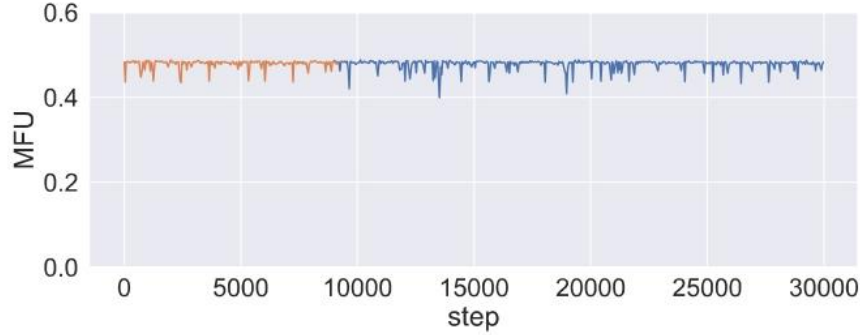


Figure 5.11: Stable MFU during production training. The MFU remains stable throughout the entire training run, showing the stability of the system.

These results demonstrate that MegaScale enables stable, efficient production training of LLMs at the scale of 10K GPUs, providing practical insights for large-scale LLM training.

5.6 Summary

This chapter presented MegaScale, a production-grade system for training LLMs on more than 10,000 GPUs through full-stack algorithm-system co-design. We began by motivating the unique challenges of production-scale training, contrasting the controlled research environment with the harsh realities of large-scale deployment where failures, stragglers, and network issues are normal rather than exceptions.

The key innovation of MegaScale lies in its holistic approach to system design. Rather than optimizing individual components in isolation, MegaScale considers the interactions between algorithmic choices, communication patterns, operator implementations, data pipelines, and network configurations. This integrated perspective enables optimizations that would be invisible to component-wise analysis, such as the communication overlapping techniques that hide communication behind computation across all three parallelism dimensions.

We detailed several critical technical contributions. The algorithmic optimizations, including parallel transformer blocks, sliding window attention, and the LAMB optimizer, improve training efficiency without sacrificing model quality. The communication overlapping techniques for data, pipeline, and tensor parallelism reduce idle time and improve MFU significantly. The operator optimizations with FlashAttention-2 and kernel fusion reduce kernel launch overhead. The network tuning for ECMP and PFC reduces congestion and improves stability.

The robust fault tolerance framework represents another essential contribution. By implementing automatic failure detection, diagnosis, and recovery, MegaScale enables training runs that span weeks while recovering from hundreds of failures. The optimized checkpointing and recovery procedures minimize the overhead of these operations, allowing training to proceed nearly uninterrupted.

The observability tools developed for MegaScale address a critical gap in large-scale training. Traditional monitoring approaches fail to capture the complex interactions between components at scale. MegaScale’s CUDA event heat maps, distributed timeline traces, and

3D parallel visualization enable rapid diagnosis of issues that would otherwise require days or weeks to resolve.

The production results speak for themselves: 55.2% MFU when training a 175B model on 12,288 GPUs, representing a 1.34x improvement over Megatron-LM. More importantly, MegaScale has demonstrated stable production operation, maintaining continuous training for weeks while recovering from over 100 failures automatically. These results establish a new benchmark for large-scale LLM training and provide a foundation for future systems targeting even larger scales.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This thesis has presented a comprehensive study of the evolution of distributed LLM training systems, tracing the progression from manual parallelism design to automated optimization and finally to production-grade deployment at extreme scales. Through detailed analysis of three representative systems—Megatron-LM, Alpa, and MegaScale—we have illuminated the key innovations, technical challenges, and practical insights that define this rapidly evolving field.

Megatron-LM made pioneering contributions to the field of distributed LLM training. First, it demonstrated that efficient tensor parallelism could be achieved through simple modifications to standard deep learning frameworks, without custom compilers or specialized hardware. Second, it introduced a communication-efficient tensor parallelism design that partitions transformer computations to minimize synchronization points, achieving only four all-reduce operations per layer. Third, it provided critical architectural insights for scaling transformers, discovering that pre-LN architecture stabilizes training for large models. Fourth, it established scaling laws demonstrating that larger models consistently achieve better performance, driving the subsequent development of models with hundreds of billions of parameters. Fifth, through open-sourcing its implementation, Megatron-LM democratized access to large-scale LLM training for the research community.

Alpa advanced the field by automating the parallelization process. Its primary contribution was the novel hierarchical view of parallelism that unifies intra-operator and inter-operator parallelism, enabling systematic exploration of the combined parallelization space that was previously inaccessible to manual design. It developed a two-level compilation approach with efficient algorithms—ILP for intra-operator optimization and DP for inter-operator optimization—that automatically finds optimal parallelization plans for arbitrary models. It implemented cross-mesh resharding optimization that enables efficient communication across device meshes with different shapes, significantly reducing communication overhead. Alpa demonstrated that automation could match or exceed hand-tuned systems while generalizing to diverse architectures including MoE models and vision transformers. By making the system publicly available, Alpa provided the research community with automated access to efficient distributed training.

MegaScale bridged the gap between research prototypes and production deployment. Its key contribution was the full-stack algorithm-system co-design approach that optimizes every component of distributed LLM training, achieving 55.2% MFU on 12,288 GPUs. It developed comprehensive communication overlapping techniques that hide communication overhead behind computation across all three parallelism dimensions, significantly improving GPU utilization. It implemented a robust fault tolerance framework with automatic failure detection, diagnosis, and recovery, enabling uninterrupted training across hundreds of failures. It built extensive observability tools including CUDA event heat maps, distributed timeline traces, and 3D parallel visualization that enable diagnosis of complex issues at scale. Finally, it shared invaluable production experience and insights that guide practitioners deploying large-scale training systems.

Together, these three systems represent the evolution of distributed LLM training from early research demonstrations to mature production infrastructure. Megatron-LM established that large-scale training was feasible, Alpa demonstrated that it could be automated, and MegaScale proved that it could run reliably in production environments.

6.2 Conclusion

In this thesis, we have presented a comprehensive integration of three representative distributed LLM training systems, which represent the key milestones in the evolution of this field. From the early manual tensor parallelism system Megatron-LM, to the automated hierarchical parallelism system Alpa, to the production-grade full-stack optimization system MegaScale, the field has evolved rapidly to address the increasing demand of large language model training.

Megatron-LM pioneered the tensor parallelism approach, enabling the first training of billion-parameter transformer models with high efficiency. It laid the foundation for all subsequent distributed LLM training systems, and demonstrated that large models could achieve significantly better performance. Alpa further advanced the field by automating the parallelization process, enabling generalizable training for arbitrary models without manual tuning, which democratized the access to large-scale training. MegaScale then brought the technology to production, enabling stable and efficient training at the scale of 10K GPUs, addressing the practical challenges of real-world deployment.

6.3 Future Work

Looking forward, the field of distributed LLM training is still evolving rapidly, driven by the ever-increasing demand for larger and more capable models. There are several promising directions for future research that build upon the foundations laid by Megatron-LM, Alpa, and MegaScale.

First, as the model size continues to grow toward trillion-parameter models and beyond, we need to scale the training to even larger clusters, with hundreds of thousands of GPUs. This will require new parallelism techniques that can efficiently utilize heterogeneous resources across geographically distributed data centers. Research on distributed training

across multiple clusters [8] and cross-region training will become increasingly important. This will also require new fault tolerance mechanisms to handle the exponentially increasing failure rate at such scales, including proactive failure prediction and intelligent checkpoint placement strategies.

Second, we need to improve the automation of the parallelization system to support dynamic models and elastic training. Current systems require significant configuration upfront, but production environments often need to adapt to changing conditions like GPU availability, network topology changes, and priority adjustments. Elastic training frameworks that can dynamically adjust parallelism degrees during training will be essential for maximizing resource utilization in shared clusters. Additionally, supporting dynamic computational graphs that change during training, such as mixture-of-experts models with conditional expert activation, will require new parallelism approaches that can adapt in real-time.

Third, we need to support heterogeneous hardware, such as different types of GPUs, CPUs, FPGAs, and AI accelerators, to utilize the heterogeneous resources available in modern data centers. The emergence of custom accelerators like Google’s TPU, Apple’s Neural Engine, and various domain-specific accelerators provides opportunities for heterogeneous training that can match different computational patterns to optimal hardware. However, this heterogeneity introduces significant challenges in workload distribution, communication optimization, and memory management. Research on heterogeneous parallelism will be crucial for fully utilizing these diverse hardware resources.

Fourth, we need to improve the observability and diagnosis tools to handle the increasing complexity of large-scale training systems. As training runs span thousands of GPUs for weeks or months, the volume of telemetry data becomes overwhelming, and traditional monitoring approaches break down. Machine learning-based anomaly detection and root cause analysis can help identify issues faster and more accurately. Visualization tools that can represent the complex interactions between parallelism dimensions, network traffic, and computational patterns will be essential for enabling practitioners to understand and optimize their training jobs.

Fifth, energy efficiency and environmental impact are becoming increasingly important considerations for large-scale training. The carbon footprint of training large models has received significant attention, with estimates suggesting that training GPT-3 emitted the equivalent of approximately 500 tons of CO₂. Future systems should incorporate energy awareness into their optimization objectives, balancing performance with sustainability. Techniques like dynamic voltage and frequency scaling (DVFS), smart resource allocation to minimize idle power, and renewable energy-aware scheduling can help reduce the environmental impact of large-scale training.

Sixth, we need to develop better benchmarking and evaluation standards for distributed training systems. Current benchmarks often focus on narrow metrics like MFU or scaling efficiency, without capturing the full picture of system performance including fault tolerance, usability, and cost efficiency. Standardized benchmarks are a step in the right direction, but more comprehensive evaluation frameworks are needed to guide system design and procurement decisions.

We believe that the evolution of distributed LLM training systems will continue to drive the development of large language models, enabling even more powerful models and applications in the future. The innovations pioneered in Megatron-LM, Alpa, and MegaScale

provide a foundation for this continued progress, and we look forward to seeing how future systems will build upon these foundations to overcome the challenges outlined above.

References

- [1] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 472–487, 2022.
- [2] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1897, 2020.
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Christopher Ré. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [4] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.
- [5] Tri Dao. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.
- [6] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [8] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. Glam: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning*, pages 5547–5569. PMLR, 2022.
- [9] Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ratul Huang, David Maltz, Zhi Liu, Vinay Wang, Hao Pang, Hui Chen, et al. Pingmesh: A large-scale system for

- data center network latency measurement and analysis. *ACM SIGCOMM Computer Communication Review*, 45(4):139–152, 2015.
- [10] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *Advances in Neural Information Processing Systems*, 35:30016–30030, 2022.
 - [11] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in Neural Information Processing Systems*, 32, 2019.
 - [12] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. *arXiv preprint arXiv:1807.05358*, 2018.
 - [13] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. Megascale: Scaling large language model training to more than 10,000 gpus. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 745–762, 2024.
 - [14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
 - [15] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
 - [16] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704*, 2020.
 - [17] Kai Liu, Zhipeng Jiang, Junchen Zhang, Hao Wei, Xiang Zhong, Lihua Tan, and Tian Pan. Hostping: Diagnosing intrahost network bottlenecks in rdma servers. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 15–29, 2023.
 - [18] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
 - [19] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Garth A Gibson, and Martin Steffen. Pipedream: Weighted

- pipeline parallelism for dnn training. In *Proceedings of the ACM SIGSOPS European Conference on Computer Systems*, pages 1–15, 2021.
- [20] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091*, 2019.
 - [21] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
 - [22] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
 - [23] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations for training and deployment of large-scale models. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3165–3166, 2020.
 - [24] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018.
 - [25] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Peter Koanantakool, Peter Hawkins, Hyoungho Lee, Mingsheng Hong, Cliff Young, et al. Mesh-tensorflow: Deep learning for supercomputers. *arXiv preprint arXiv:1811.02084*, 2018.
 - [26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
 - [27] Chentao Tan, Zehua Jin, Chuanxiong Guo, Ting Zhang, Hao Wu, Kai Deng, and Dong Bi. Netbouncer: Active device and link failure localization in data center networks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 599–613, 2019.
 - [28] TensorFlow Authors. Xla: Tensorflow’s unified compiler. Google Research Blog, 2017. <https://developers.googleblog.com/2017/10/xla-tensorflows-unified-compiler.html>.
 - [29] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
 - [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

- [31] Minjie Wang, Chien-chin Huang, and Jinyang Li. Supporting very large models using automatic dataflow graph partitioning. In *Proceedings of the Fourteenth EuroSys Conference 2019*, pages 1–17, 2019.
- [32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [33] Fanlong Zeng, Wensheng Gan, Yongheng Wang, and Philip S Yu. Distributed training of large language models: A survey. *Natural Language Processing Journal*, page 100174, 2025.
- [34] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. Alpa: Automating inter-and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, 2022.